

Master in Statistics for Data Science  
2025-2026

*Master Thesis*

# Forecasting in the Italian day-ahead electricity market

---

María Colado Montañés

Andrés M. Alonso Fernández  
Madrid, July 2026

## AVOID PLAGIARISM

The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

## SUMMARY

This thesis studies day-ahead forecasting in the Italian electricity market, focusing on the seven physical bidding zones of the *Mercato del Giorno Prima* (MGP). The objective is to forecast three related hourly market outcomes: zonal electricity prices, accepted purchase and sale volumes. The dataset combines historical market data from Gestore dei Mercati Energetici (GME) for the period 2021–2025 with calendar information and zonal weather variables. The empirical analysis compares statistical models, tree-based machine learning methods and deep learning architectures under a common evaluation framework. In particular, SARIMAX, LEAR, Random Forest, XGBoost, LSTM and TCN are evaluated, using relative MAE against a weekly seasonal naive benchmark as the main ranking metric. The results show that tree-based machine learning models provide the strongest performance. Overall, the findings highlight the importance of daily and weekly patterns, zonal heterogeneity and lagged market interactions in the Italian day-ahead electricity market.

**Keywords:** Day-ahead electricity market, Machine learning, Time series forecasting, Deep learning, Italian electricity market.

# Contents

---

|          |                                                      |           |
|----------|------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                  | <b>1</b>  |
| <b>2</b> | <b>Data and Exploratory Analysis</b>                 | <b>3</b>  |
| 2.1      | Data extraction . . . . .                            | 3         |
| 2.1.1    | Market data . . . . .                                | 3         |
| 2.1.2    | Weather data . . . . .                               | 4         |
| 2.2      | Data preprocessing . . . . .                         | 6         |
| 2.3      | Exploratory data analysis . . . . .                  | 6         |
| <b>3</b> | <b>Methodology</b>                                   | <b>10</b> |
| 3.1      | Experimental Framework . . . . .                     | 12        |
| 3.1.1    | Benchmark models . . . . .                           | 12        |
| 3.1.2    | Evaluation metrics and statistical testing . . . . . | 13        |
| 3.2      | SARIMAX . . . . .                                    | 15        |
| 3.2.1    | Theoretical background . . . . .                     | 15        |
| 3.2.2    | Implementation in this study . . . . .               | 18        |
| 3.3      | LEAR . . . . .                                       | 19        |
| 3.3.1    | Theoretical background . . . . .                     | 19        |
| 3.3.2    | Implementation in this study . . . . .               | 22        |
| 3.4      | Random Forest . . . . .                              | 23        |
| 3.4.1    | Theoretical background . . . . .                     | 23        |
| 3.4.2    | Implementation in this study . . . . .               | 24        |
| 3.5      | XGBoost . . . . .                                    | 26        |
| 3.5.1    | Theoretical background . . . . .                     | 26        |
| 3.5.2    | Implementation in this study . . . . .               | 26        |
| 3.6      | LSTM . . . . .                                       | 27        |
| 3.6.1    | Theoretical background . . . . .                     | 27        |

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| 3.6.2    | Implementation in this study . . . . .           | 29        |
| 3.7      | Temporal Convolutional Network . . . . .         | 31        |
| 3.7.1    | Theoretical background . . . . .                 | 31        |
| 3.7.2    | Implementation in this study . . . . .           | 32        |
| <b>4</b> | <b>Forecast results</b>                          | <b>35</b> |
| 4.1      | SARIMAX . . . . .                                | 35        |
| 4.2      | LEAR . . . . .                                   | 37        |
| 4.3      | Random Forest . . . . .                          | 38        |
| 4.4      | XGBoost . . . . .                                | 41        |
| 4.5      | LSTM . . . . .                                   | 42        |
| 4.6      | Temporal Convolutional Network . . . . .         | 44        |
| 4.7      | Final model comparison . . . . .                 | 45        |
| <b>5</b> | <b>Conclusions</b>                               | <b>48</b> |
|          | <b>Bibliography</b>                              | <b>50</b> |
| <b>A</b> | <b>Additional results</b>                        |           |
| A.1      | Weekly persistence diagnostics . . . . .         |           |
| A.2      | XGBoost feature-importance diagnostics . . . . . |           |
| A.3      | Final model comparison results . . . . .         |           |
|          | Declaration of Generative AI . . . . .           |           |

# List of Figures

---

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Zonal configuration of the Italian electricity market. Source: [5], Figure 1.                                                                                                                                                                                                                                                                                                                                                                                    | 1  |
| 2.1 | Monthly average electricity prices by bidding zone. . . . .                                                                                                                                                                                                                                                                                                                                                                                                      | 7  |
| 2.2 | Monthly behaviour of purchases and sales in representative zones. . . . .                                                                                                                                                                                                                                                                                                                                                                                        | 8  |
| 2.3 | Average intraday price profiles on weekdays and weekends. . . . .                                                                                                                                                                                                                                                                                                                                                                                                | 8  |
| 2.4 | Autocorrelation by target in NORD. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                       | 9  |
| 3.1 | SARIMAX rolling forecasting scheme. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                      | 18 |
| 3.2 | LEAR scheme based on the implemented forecasting pipeline. . . . .                                                                                                                                                                                                                                                                                                                                                                                               | 23 |
| 3.3 | Main components of the Temporal Convolutional Network architecture. Panel (a) shows a stack of causal dilated convolutional layers with increasing dilation factors. Panel (b) shows the structure of a residual block, including dilated causal convolutions, weight normalization, ReLU activations, dropout and the optional $1 \times 1$ convolution used to match dimensions. Panel (c) illustrates the residual connection inside a block. Source: [16]. . | 32 |
| 4.1 | Autocorrelation function of SARIMAX residuals for purchases in NORD.                                                                                                                                                                                                                                                                                                                                                                                             | 36 |
| 4.2 | Aggregated impurity-based Random Forest feature importance by target. Shares are normalised within each target. . . . .                                                                                                                                                                                                                                                                                                                                          | 40 |
| 4.3 | Final model comparison on the 2025 test set. The dashed line represents the weekly seasonal naive benchmark. . . . .                                                                                                                                                                                                                                                                                                                                             | 46 |
| 4.4 | Model performance by target and zone on the 2025 test set. Values below one indicate improvement over the weekly seasonal naive benchmark. . .                                                                                                                                                                                                                                                                                                                   | 47 |
| A.1 | Top 10 individual XGBoost features by target, ranked by aggregated gain.                                                                                                                                                                                                                                                                                                                                                                                         |    |

# List of Tables

---

|      |                                                                                                                                               |    |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1  | Main market variables used in the final dataset. . . . .                                                                                      | 4  |
| 2.2  | Representative cities used to retrieve weather data by GME zone. . . . .                                                                      | 5  |
| 2.3  | Descriptive statistics of the target variables in the full hourly panel. . . . .                                                              | 7  |
| 3.1  | Fixed SARIMA error specifications used in the final SARIMAX implementation. . . . .                                                           | 19 |
| 3.2  | Candidate multi-output structures considered for Random Forest. . . . .                                                                       | 25 |
| 3.3  | Summary of the LSTM specifications considered. . . . .                                                                                        | 30 |
| 3.4  | Summary of the TCN specifications considered. . . . .                                                                                         | 34 |
| 4.1  | SARIMAX strategy selected in validation by target. . . . .                                                                                    | 35 |
| 4.2  | SARIMAX performance on the 2025 test set, averaged across zones. . . . .                                                                      | 35 |
| 4.3  | LEAR strategy selected in validation by target. . . . .                                                                                       | 37 |
| 4.4  | LEAR performance on the 2025 test set, averaged across zones. . . . .                                                                         | 37 |
| 4.5  | Distribution of non-zero LEAR coefficients on the 2025 test set. . . . .                                                                      | 38 |
| 4.6  | Random Forest structure selected in validation by target. . . . .                                                                             | 38 |
| 4.7  | Random Forest performance on the 2025 test set, averaged across zones. . . . .                                                                | 39 |
| 4.8  | XGBoost strategy selected in validation by target. . . . .                                                                                    | 41 |
| 4.9  | XGBoost performance on the 2025 test set, averaged across zones. . . . .                                                                      | 41 |
| 4.10 | Direct expanding LSTM architecture selected in validation by target. . . . .                                                                  | 43 |
| 4.11 | LSTM performance on the 2025 test set, averaged across zones. . . . .                                                                         | 43 |
| 4.12 | TCN performance on the 2025 test set, averaged across zones. . . . .                                                                          | 44 |
| 4.13 | Best model by target on the 2025 test set, averaged across zones. . . . .                                                                     | 45 |
| A.1  | Weekly persistence diagnostics by zone in the 2025 test period. Correlations are computed with the same variable lagged by 168 hours. . . . . |    |
| A.2  | Final test performance by model and target, averaged across zones. . . . .                                                                    |    |

A.3 Summary of zone-level Diebold–Mariano tests against the weekly seasonal naive benchmark. Each row summarizes the seven tests obtained for the seven physical bidding zones. . . . .

A.4 Complete test performance by model, target and zone. . . . .

# 1. Introduction

---

Electricity is unlike most commodities. It is economically non-storable, power system security requires production and consumption to be balanced continuously, and electricity demand is strongly affected by weather conditions and patterns of economic and social activity. These features generate price dynamics with pronounced daily, weekly and annual seasonality, high volatility and occasional spikes, making forecasting both technically challenging and economically relevant [1]. Accurate predictions of prices, demand and supply are therefore critical inputs for generators, retailers, system operators and regulators, since they inform bidding strategies, risk management, operational planning, and the efficient use of generation and transmission resources [1], [2].

Within Europe, the Italian day-ahead electricity market (Mercato del Giorno Prima, MGP) presents a particularly rich forecasting environment. Managed by the Gestore dei Mercati Energetici (GME), the MGP is the central auction based market in Italy, where electricity is traded for delivery on the following day [3], [4]. Italy, however, is not a single, homogeneous market. Due to transmission constraints, regional differences in generation and consumption, and cross-border interconnections, the Italian market is organised into seven physical bidding zones: NORD, CNOR, CSUD, SUD, CALA, SICI, and SARD [4]. As a result, market outcomes may differ substantially across zones and hours. The national reference price, the *Prezzo Unico Nazionale* (PUN), provides a useful aggregate benchmark, but it hides regional heterogeneity that is relevant for zonal forecasting and market analysis [4].

This thesis focuses on forecasting in the Italian day-ahead market, treating each of the seven bidding zones as a distinct forecasting unit. For each zone, three variables are modelled simultaneously: the zonal clearing price, total purchase volumes (demand bids), and total sale volumes (supply offers). Modelling purchases and sales alongside prices is motivated by a straightforward economic logic, price formation in each zone reflects the intersection of the local bid, supply offers and network constraints, so changes in volume (possibly caused by industrial demand cycles, renewable generation and seasonal factors)



**Fig. 1.1.** Zonal configuration of the Italian electricity market. Source: [5], Figure 1.



are not merely correlates of price but its structural determinants. The joint forecasting of the three elements allows each model to explicitly take advantage of these interdependencies.

The dataset used throughout this work is from historical GME records and covers the period 2021-2025 of hourly market data across all seven bidding zones. Weather information is also included, since meteorological conditions affect electricity consumption patterns and can indirectly influence market outcomes through changes in demand and renewable generation.

A suite of complementary forecasting methods is developed and benchmarked to capture the range of dynamics present in the data, from seasonal patterns and intraday price profiles to nonlinear interactions among zones and between electricity prices and weather variables. These span classical statistical models (LEAR, SARIMAX), tree-based machine learning (Random Forest, XGBoost), and deep learning architectures (LSTM, TCN). Together, they offer a systematic comparison across modelling paradigms, with the Italian zonal market as a consistent, well-defined experimental setting.

The remainder of this thesis is structured as follows. Chapter 2 describes the data sources, the preprocessing decisions, and the construction of the modelling panel. Chapter 3 presents the forecasting framework, including the benchmark models, the statistical, machine learning and deep learning specifications. Chapter 4 reports and discusses the empirical results of the different models across targets and zones. Finally, Chapter 5 summarizes the main findings, limitations and possible extensions. The scripts used in this thesis are available in the project repository.<sup>1</sup>

---

<sup>1</sup>[https://github.com/mariacolmon/Forecasting\\_Italian\\_day\\_ahead\\_electricity\\_market](https://github.com/mariacolmon/Forecasting_Italian_day_ahead_electricity_market)

## 2. Data and Exploratory Analysis

---

This chapter describes the data sources, preprocessing decisions, and exploratory evidence used to construct the final modelling dataset. Since the objective of this thesis is to forecast day-ahead electricity prices, purchases and sales in the Italian market, the final dataset combines historical market outcomes from GME with zonal weather variables, both of which are relevant for modelling electricity prices and traded volumes [1], [2].

### 2.1. Data extraction

#### 2.1.1. Market data

Market data is obtained from historical records published by Gestore dei Mercati Energetici (GME) for the *Mercato del Giorno Prima* (MGP) [3], [6]. The raw information is provided in yearly Excel files containing several sheets with prices, purchase volumes, sale volumes and complementary market indicators.

As it has already been established in the Introduction (Chapter 1), the empirical analysis is restricted to the seven physical Italian bidding zones: NORD, CNOR, CSUD, SUD, CALA, SICI and SARD. The data provided by GME also contain external and virtual areas information, but they are not treated as additional forecasting units. Instead, their information is retained through aggregate variables, such as total external purchase and sale volumes and the number of active external areas.

The final market dataset covers the period 2021–2025 and is restricted to the hourly regime. Although the Italian day-ahead market moved to a quarter-hourly structure in October 2025, the main empirical analysis is conducted on a consistent hourly dataset.

The three forecasted variables are the zonal electricity price, the accepted purchase volume and the accepted sale volume. They are denoted respectively as `price`, `purchases` and `sales`. The price is measured in EUR/MWh, whereas purchases and sales are measured in MWh.

Several additional market variables are retained as predictors. The Herfindahl-Hirschman Index, denoted as `hhi`, is used as an indicator of market concentration. The residual supply indicator, denoted as `rsi`, provides information on residual supply conditions in the market. The marginal technology indicator, denoted as `mti`, identifies the technology used for power generation [4].

The dataset also includes national market variables. The PUN Index GME, denoted

as pun, is the national reference price index derived from zonal market outcomes [7]. National purchase, sale and unsold volumes are retained as common predictors and denoted as `purchases_italy`, `sales_italy` and `unsold_italy`. These variables provide information on the aggregate Italian market. However, they are used only through lagged values available before the forecast origin, since their contemporaneous values would not be known when producing a day-ahead forecast. Table 2.1 summarises the main market variables used in the final modelling dataset.

| Variable                     | Role      | Description                                                                           |
|------------------------------|-----------|---------------------------------------------------------------------------------------|
| <code>price</code>           | Target    | Zonal day-ahead electricity price, measured in EUR/MWh.                               |
| <code>purchases</code>       | Target    | Accepted purchase volume in the corresponding bidding zone, measured in MWh.          |
| <code>sales</code>           | Target    | Accepted sale volume in the corresponding bidding zone, measured in MWh.              |
| <code>hhi</code>             | Predictor | Herfindahl-Hirschman Index, used as an indicator of market concentration.             |
| <code>rsi</code>             | Predictor | Residual supply indicator, used to describe residual supply conditions in the market. |
| <code>mti</code>             | Predictor | Marginal technology indicator, treated as a categorical market-structure variable.    |
| <code>pun</code>             | Predictor | National reference price index derived from zonal market outcomes.                    |
| <code>purchases_italy</code> | Predictor | Total Italian purchase volume, used as a national market-level explanatory variable.  |
| <code>sales_italy</code>     | Predictor | Total Italian sale volume, used as a national market-level explanatory variable.      |
| <code>unsold_italy</code>    | Predictor | Total Italian unsold volume, used to retain information on unmatched supply.          |

**TABLE 2.1.** Main market variables used in the final dataset.

### 2.1.2. Weather data

Weather variables are included because the relationship between electricity markets and meteorological conditions is well established in the electricity forecasting literature. Weron [1] notes that electricity demand depends on weather conditions, such as temperature, wind speed and precipitation, as well as on patterns of business and everyday activity. These factors contribute to the strong daily, weekly and annual seasonality, high volatility and

occasional spikes observed in electricity prices.

Historical meteorological data are obtained from the Open-Meteo Historical Weather API, which provides historical weather data at hourly resolution for geographical coordinates [8]. For this thesis, weather data are downloaded for a set of representative cities within each Italian physical bidding zone and then aligned with the hourly GME market panel. The representative cities used for each GME physical zone are reported in Table 2.2. The variables retrieved from the API are:

`temperature_2m,`      `wind_speed_100m,`      `shortwave_radiation.`

Temperature at 2 metres is used as a proxy for heating and cooling-related electricity demand. Wind speed at 100 metres is used as a proxy for wind generation conditions, which affect the supply side of the electricity market. Shortwave radiation is used as a proxy for solar generation conditions, especially during daylight hours.

| GME zone | Representative cities       |
|----------|-----------------------------|
| NORD     | Milano, Torino, Venezia.    |
| CNOR     | Bologna, Firenze, Ancona.   |
| CSUD     | Roma, Pescara, Napoli.      |
| SUD      | Bari, Foggia, Potenza.      |
| CALA     | Catanzaro, Reggio Calabria. |
| SICI     | Palermo, Catania.           |
| SARD     | Cagliari, Sassari.          |

**TABLE 2.2.** Representative cities used to retrieve weather data by GME zone.

For each weather variable, the city-level observations are averaged across the selected cities belonging to the same GME zone and hour. The resulting zonal weather series contains one hourly value per zone and can therefore be joined directly to the market panel by zone, date and hour.

The resulting weather dataset is joined to the market data by zone and hourly timestamp. In the forecasting experiments, delivery-day weather variables are interpreted as weather forecast proxies. This assumption is consistent with an operational day-ahead forecasting setting, where weather forecasts for the delivery day are normally available before market delivery. This approach has been used in other works, for example, by Huurman et al. in their study on Scandinavian day-ahead electricity prices [9].

## 2.2. Data preprocessing

After extracting the market and weather data, several preprocessing steps are applied to obtain a consistent hourly panel for forecasting.

Daylight-saving time transitions are normalised to obtain a standard 24-hour local daily grid. Following the preprocessing approach used by [10], duplicated local hours in the autumn transition are collapsed into a single observation by averaging the duplicated numerical values, while missing local hours in the spring transition are filled by linear interpolation using neighbouring hours. After this step, each date–zone combination contains exactly 24 hourly observations.

The zonal weather series are then joined to the market panel by zone, date and hour. This produces the final modelling dataset, which contains the target variables, market indicators, national market variables, external-area aggregates and zonal weather predictors in a common hourly structure.

A small set of calendar variables is also created to capture systematic temporal patterns: year, month, day of the week, hour of the day and a weekend indicator. These variables are included because electricity prices and traded volumes display strong intraday and weekly regularities, as shown in the exploratory analysis below.

Finally, quality-control checks are performed on the final panel. These checks verify the absence of duplicated date–hour–zone records and the completeness of the 24-hour daily structure. The main numerical variables used for modelling contain no missing values. The only missingness concerns the categorical variable `mti`, with 169 missing observations (approximately 0.055%). These will be handled by imputation using the mode. No negative purchase or sale volumes are detected. The `hhi` values lie within the expected range from 0 to 10,000. The `rsi` variable is highly skewed, with some large values, and is therefore retained but treated with caution.

## 2.3. Exploratory data analysis

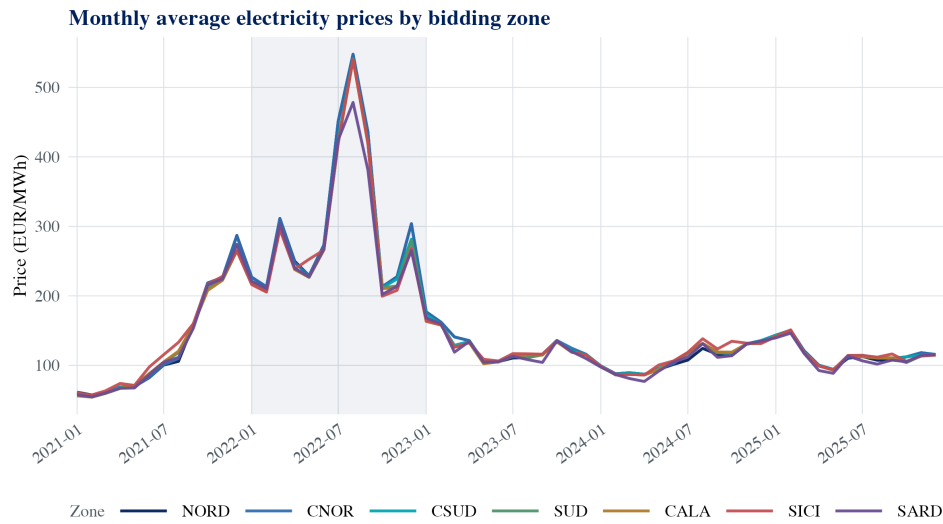
This section provides a concise exploratory analysis of the dataset. The objective is to identify strong temporal variation, differences across targets and zones, calendar effects, etc.

Table 2.3 reports summary statistics for the three forecasting targets.

| Target    | N       | Missing | Mean     | Std. dev. | Median   | P05    | P95       |
|-----------|---------|---------|----------|-----------|----------|--------|-----------|
| Price     | 306,768 | 0       | 154.66   | 103.52    | 121.60   | 54.67  | 377.62    |
| Purchases | 306,768 | 0       | 4,568.14 | 5,912.17  | 2,107.20 | 576.28 | 20,159.12 |
| Sales     | 306,768 | 0       | 3,789.85 | 4,419.33  | 2,012.00 | 779.11 | 15,316.55 |

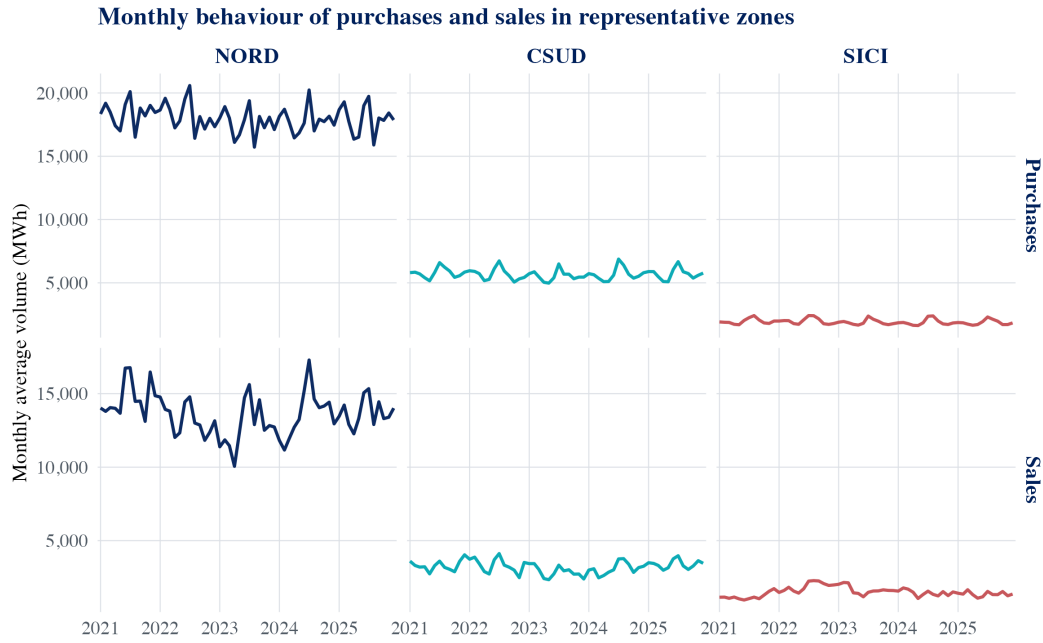
**TABLE 2.3.** Descriptive statistics of the target variables in the full hourly panel.

Figure 2.1 shows the monthly average day-ahead electricity price for the seven physical bidding zones. The most visible feature is the sharp increase in prices during 2022, followed by a gradual decline during 2023 and more moderate levels in 2024–2025. This episode coincides with the European energy crisis triggered by Russia’s invasion of Ukraine and the associated surge in natural gas prices, which strongly affected wholesale electricity prices across Europe [11]. The shock is common to all zones, indicating that the dominant movement during this period is a market-wide price episode rather than an isolated regional pattern.



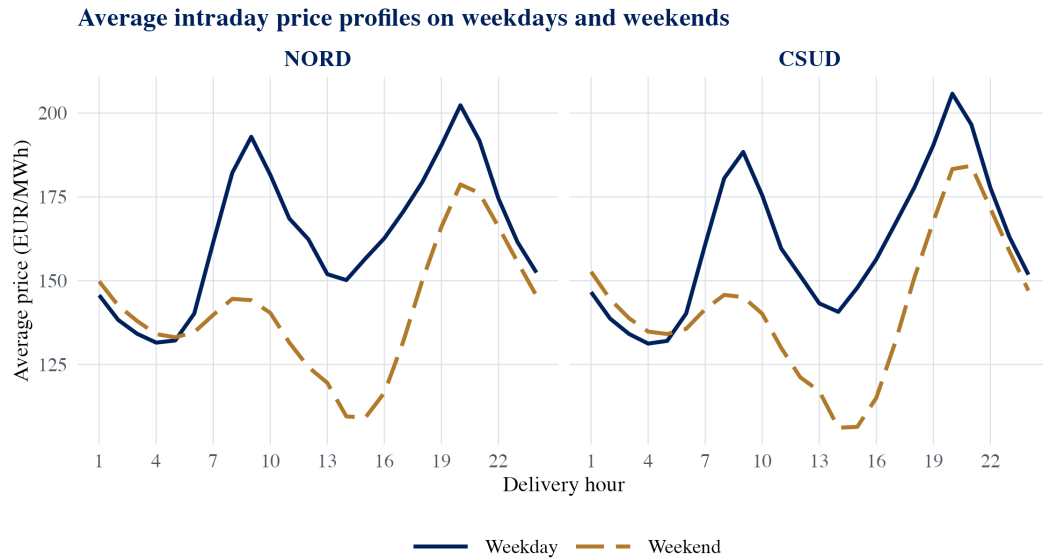
**Fig. 2.1.** Monthly average electricity prices by bidding zone.

The behaviour of purchase and sale volumes is illustrated in Figure 2.2 for three representative zones. The figure shows that volume targets differ substantially in scale across zones, with NORD representing a much larger market than CSUD and SICI. Purchases and sales also present different temporal dynamics within the same zone.



**Fig. 2.2.** Monthly behaviour of purchases and sales in representative zones.

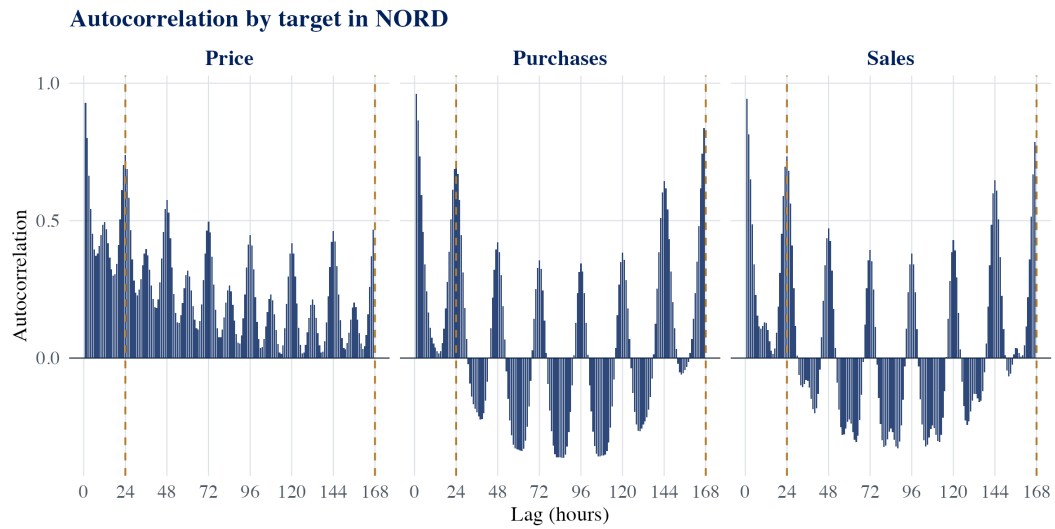
Calendar effects are explored in Figure 2.3, which compares average hourly price profiles for weekdays and weekends in two representative zones. The profiles show a pronounced intraday structure, with higher prices during daytime and evening hours. Weekend profiles are generally lower and have a different daily shape.



**Fig. 2.3.** Average intraday price profiles on weekdays and weekends.

Finally, Figure 2.4 reports autocorrelation functions for the three targets in NORD, using lags up to one week. The three series display substantial serial dependence, with recurrent peaks at daily and weekly lags. Rather than resembling independent noise, the targets

exhibit a clear seasonal time-series structure as expected.



**Fig. 2.4.** Autocorrelation by target in NORD.

Overall, the exploratory analysis shows that the dataset combines market-wide shocks, zonal heterogeneity, calendar regularities and strong serial dependence. These features are taken into account in the modelling framework developed in this project.



### 3. Methodology

---

Electricity price forecasting (EPF) developed as a distinct research field around the turn of the century, following the liberalization of electricity markets in Europe, North America, and Australia. Once electricity prices began to be determined through competitive market mechanisms rather than regulated tariffs, market participants faced a forecasting problem with features that were unusual in standard commodity markets. Electricity is economically non-storable, system operation requires a continuous balance between production and consumption, and prices are shaped by a dense interaction of demand patterns, generation availability, fuel costs, weather conditions and market behaviour. As a result, electricity prices display strong daily, weekly and annual seasonalities, abrupt spikes, high volatility and a pronounced dependence on calendar and fundamental variables [1].

Early EPF research relied heavily on statistical and econometric time-series methods, adapting AR, ARMA, ARIMA and ARX-type models to the specific dynamics of electricity prices. Over the following two decades, the field broadened considerably. Regularized autoregressive models were proposed to handle large sets of lagged and exogenous predictors; machine learning methods were increasingly adopted to capture nonlinear relationships; and, more recently, deep learning architectures have been applied to model complex temporal structures in high-frequency electricity data [1], [2].

This proliferation of methods, however, has not been matched by a full consensus on best practice. Weron [1], in a comprehensive review of the field, argues that EPF studies should be evaluated through objective comparative designs using the same datasets, robust error measures and statistical tests of predictive accuracy. Lago et al. [2] extend this critique by showing that many published comparisons are difficult to interpret because they rely on short test periods, non-public datasets, weak benchmarks, inconsistent metrics or insufficiently documented experimental settings. These concerns motivate the methodological design of this thesis. Rather than developing a single forecasting model in isolation, the analysis adopts a comparative forecasting framework in which several models from different methodological traditions are evaluated under a common protocol, where the same forecasting targets, bidding zones, horizons, information set and out-of-sample evaluation periods are used across all models.

Although most of the methodological literature reviewed in this chapter is framed around electricity price forecasting specifically, the forecasting task considered in this thesis is broader. The same modelling families are applied not only to prices, but also to day-ahead purchases and sales in each bidding zone. This extension is natural. As discussed in the introduction, prices and traded volumes are jointly determined by diverse factors like demand, supply conditions, weather, calendar effects, or regional market

structure. The methods reviewed are developed precisely to model series with these characteristics, regardless of whether the series in question is a price or a volume.

The model selection is inspired by two central references in the EPF literature. Weron [1] distinguishes between statistical time-series approaches and computational intelligence methods, while Lago et al. [2] update this perspective by explicitly reviewing modern statistical, machine learning and deep learning approaches for day-ahead electricity forecasting.

**Statistical models.** The first family includes models that explicitly represent temporal dependence and seasonality. SARIMAX extends the classical ARIMA framework by incorporating seasonal components and exogenous regressors, making it a natural benchmark for hourly electricity data, where the autocorrelation structure is strongly influenced by daily and weekly cycles (see, for instance, [12]). In parallel, this thesis includes the LASSO-Estimated AutoRegressive model (LEAR), a regularized autoregressive specification that has become a strong benchmark in modern EPF studies. LEAR combines a rich set of lagged prices, exogenous variables and calendar effects with a regularized estimation procedure, allowing automatic variable selection in high-dimensional linear models [2]. The idea of including both SARIMAX and LEAR is to compare two statistical approaches with different modelling philosophies. While SARIMAX imposes an explicit seasonal autoregressive-moving-average structure, LEAR relies on a large lag-based feature set and lets regularization determine the relevant predictors.

**Tree-based machine learning models.** The second family consists of ensemble tree-based methods, in particular for this work the methods Random Forest and XGBoost. Random Forest combines many decorrelated regression trees through bagging and random feature selection, providing a robust nonlinear benchmark with relatively limited tuning requirements [13]. XGBoost, by contrast, is based on gradient-boosted decision trees and is designed to build an additive ensemble of weak learners in a sequential and regularized way [14]. These models are well suited to tabular forecasting designs because they can capture nonlinear effects and interactions between lagged market variables, calendar indicators and exogenous predictors without requiring those interactions to be specified manually. In this thesis, they serve as nonlinear alternatives to the statistical models, while using a comparable feature representation.

**Deep learning models.** The third and final family consists of neural network models designed for sequential forecasting, in particular Long Short-Term Memory networks (LSTM) and Temporal Convolutional Networks (TCN). LSTM models belong to the family of recurrent neural networks and are designed to learn temporal dependencies by updating an internal memory as the sequence is processed [15]. TCNs, by contrast, use convolutional filters over time, allowing the model to extract temporal patterns at

different lags while preserving the causal structure of the forecasting problem [16]. Both architectures are included because they provide a more flexible way of modelling complex temporal dynamics than linear autoregressive specifications, and they may capture patterns that are difficult to represent through manually defined lagged features alone.

Each family involves a different trade-off. Statistical models are generally more interpretable but may be too restrictive, tree-based methods can capture nonlinear effects but depend heavily on the quality of the feature representation and deep learning models offer greater flexibility although at the cost of higher computational complexity and lower interpretability.

The aim is not only to identify the model with the lowest forecast error, but also to evaluate the trade-off between predictive accuracy, interpretability, computational cost and robustness. If a simple and interpretable model performs competitively with more flexible alternatives, this is itself an informative result. Conversely, systematic gains from machine learning or deep learning models would suggest that nonlinear or sequential structures are relevant for forecasting the Italian zonal electricity market.

## 3.1. Experimental Framework

### 3.1.1. Benchmark models

Two naive benchmark models are used as reference forecasts throughout the empirical analysis. These benchmarks are intentionally simple and transparent, which makes them useful baselines for assessing whether more complex forecasting methods add predictive value beyond the strongest recurring patterns in the data. Let  $y_{q,z,d,h}$  denote the observed value of target  $q$  in zone  $z$ , delivery day  $d$ , and hour  $h$ , where  $q \in \{\text{price, purchases, sales}\}$  and  $h = 1, \dots, 24$ .

The first benchmark is the **day-before naive model**:

$$\hat{y}_{q,z,d,h}^D = y_{q,z,d-1,h}.$$

This forecast assigns to each hour of the delivery day the value observed at the same hour of the previous day. It is designed to capture short-term persistence and the stable intra-day structure commonly observed in electricity market variables.

The second benchmark is the **weekly seasonal naive model**:

$$\hat{y}_{q,z,d,h}^W = y_{q,z,d-7,h}.$$

This forecast uses the value observed at the same hour of the same weekday in the previous week. Its inclusion is motivated by the benchmark design discussed in [2], where the weekly naive forecast is used to account for weekly effects in day-ahead electricity price

forecasting. In the present thesis, the same idea is extended to the three target variables considered, prices, purchases and sales.

In practice, since the dataset is represented on a Daylight Saving Time (DST)-normalized hourly modelling grid, the two benchmarks are implemented as lagged forecasts with lags of 24 and 168 hours, respectively. They are computed for every target, zone, delivery day and hourly horizon included in the common evaluation index.

### 3.1.2. Evaluation metrics and statistical testing

The empirical comparison of forecasting models is based on a common set of out-of-sample error metrics. All models are evaluated on the same delivery dates, bidding zones, targets and hourly horizons. This is important, as previous reviews have stressed the need for common datasets, robust evaluation procedures, meaningful benchmarks and statistical testing of predictive accuracy [1], [2].

Let  $y_i$  denote the observed value and  $\hat{y}_i^{(m)}$  the forecast produced by model  $m$  for observation  $i$ , where  $i$  indexes a specific combination of target, zone, delivery day and hourly horizon. For a given evaluation set,  $\mathcal{I}$ , typically defined for a fixed target, zone or horizon, the forecast error is defined as

$$e_i^{(m)} = y_i - \hat{y}_i^{(m)}.$$

The first metric considered is the Mean Absolute Error (MAE):

$$\text{MAE}^{(m)} = \frac{1}{n} \sum_{i \in \mathcal{I}} |y_i - \hat{y}_i^{(m)}|,$$

where  $n = |\mathcal{I}|$ . MAE is expressed in the same units as the target variable (EUR/MWh for prices and MWh for purchases and sales). It is therefore directly interpretable as the average absolute forecasting error. What makes absolute-error metrics particularly useful is that they are less dominated by extreme observations than squared-error metrics, which makes them suitable as the basis for the main comparison.

The main ranking criterion in this thesis is the relative Mean Absolute Error (rMAE). As discussed in Section 3.1.1, the weekly seasonal naive benchmark is used as the reference model. Therefore, for each model  $m$ , rMAE is defined as

$$\text{rMAE}^{(m)} = \frac{\sum_{i \in \mathcal{I}} |y_i - \hat{y}_i^{(m)}|}{\sum_{i \in \mathcal{I}} |y_i - \hat{y}_i^{(b)}|},$$

where  $\hat{y}_i^{(b)}$  denotes the forecast from the weekly seasonal naive benchmark. Under this definition, the benchmark has  $\text{rMAE} = 1$ . Values below one indicate an improvement over the benchmark, while values above one indicate worse performance. Equivalently, the percentage improvement over the benchmark can be written as

$$\text{Improvement}^{(m)} = 100 \left( 1 - \text{rMAE}^{(m)} \right).$$

This metric is used as the main criterion for ranking models across prices, purchases and sales, since it provides a common benchmark-adjusted scale.

The Root Mean Squared Error (RMSE) is also reported:

$$\text{RMSE}^{(m)} = \sqrt{\frac{1}{n} \sum_{i \in \mathcal{I}} \left( y_i - \hat{y}_i^{(m)} \right)^2}.$$

RMSE gives larger weight to large forecast errors and is therefore useful as a complementary diagnostic, especially in electricity markets where price spikes or large volume deviations may be relevant. However, because it is more sensitive to extreme observations, it is not used as the main ranking criterion.

For purchases and sales, normalized versions of MAE and RMSE are also reported:

$$\text{NMAE}^{(m)} = \frac{\text{MAE}^{(m)}}{\frac{1}{n} \sum_{i \in \mathcal{I}} |y_i|}, \quad \text{NRMSE}^{(m)} = \frac{\text{RMSE}^{(m)}}{\frac{1}{n} \sum_{i \in \mathcal{I}} |y_i|}.$$

These metrics are useful for volume variables because purchases and sales can differ substantially in magnitude across bidding zones. By normalizing the error by the average absolute observed value, NMAE and NRMSE provide scale-adjusted measures that facilitate comparisons across series with different levels. Similar normalized error measures are commonly reported in energy forecasting applications [17], [18]. In this thesis, however, they are used only as complementary metrics for purchases and sales. The main ranking criterion remains rMAE for all targets, while MAE and RMSE in the original units are kept to preserve the direct interpretation of the forecast error.

Finally, error metrics are complemented with the Diebold–Mariano test [19]. Point estimates of MAE, RMSE or rMAE indicate which model has a lower average error, but they do not show whether the observed difference is statistically meaningful. Given two competing forecasts,  $m_1$  and  $m_2$ , the loss differential is defined as

$$d_i = L\left(e_i^{(m_1)}\right) - L\left(e_i^{(m_2)}\right),$$

where  $L(\cdot)$  is a loss function. In this thesis, the absolute-error loss,

$$L(e) = |e|,$$

is used so that the test is consistent with MAE and rMAE. The null hypothesis is

$$H_0 : \mathbb{E}(d_i) = 0,$$

which means that both forecasts have equal predictive accuracy. Rejection of the null hypothesis provides evidence that the difference in forecasting performance is statistically significant. The test is applied mainly to compare each model against the weekly seasonal naive benchmark and, when relevant, to compare the best-performing models with each other.

In addition to forecast accuracy, the computational cost of each model is recorded. This is only used as a supplementary criterion for interpreting empirical results. In an operational day-ahead forecasting setting, a model with slightly better accuracy may be less attractive if it requires substantially longer recalibration times or is difficult to update regularly. Therefore, this thesis reports, when available, total computation time, number of recalibrations and average time per forecast day. These quantities are used to assess whether the predictive gains of more complex models justify their additional computational burden.

## 3.2. SARIMAX

### 3.2.1. Theoretical background

The theoretical formulation of the SARIMAX model used in this thesis follows the notation and development of Shumway and Stoffer [20]. Let

$$\{y_t\}_{t=1}^T$$

denote a univariate time series. In this thesis,  $y_t$  represents one target variable observed at a given bidding zone, for instance  $y_t = \text{price}_{z,t}$ , where  $z$  denotes one of the Italian bidding zones. The same notation is used for purchases and sales.

The defining feature of time series data is that observations close in time are typically dependent, which makes standard methods based on independent observations unsuitable without further modelling of temporal dependence.

Let first introduce the concept of weak stationarity. Following Shumway and Stoffer [20], a time series is weakly stationary if its mean is constant and its autocovariance depends only on the lag between observations:

$$\mathbb{E}[y_t] = \mu, \quad \text{Cov}[y_t, y_{t-h}] = \gamma(h), \forall h \neq 0.$$

This assumption is clearly restrictive in electricity markets, where prices and volumes may be affected by shocks, regime changes, volatility and strong seasonal patterns. Nevertheless, it provides the mathematical starting point for ARMA and ARIMA models.

Let  $B$  denote the backshift operator,

$$By_t = y_{t-1}.$$

An ARMA  $(p, q)$  model can be written as

$$\phi(B)y_t = \theta(B)w_t,$$

where  $w_t$  is a white noise process (a stationary time series with mean zero and variance  $\sigma^2$ ), and

$$\phi(B) = 1 - \phi_1 B - \dots - \phi_p B^p, \quad \text{and} \quad \theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$$

are polynomials in  $B$ .

The autoregressive polynomial  $\phi(B)$  captures dependence on past values of the series, while the moving-average polynomial  $\theta(B)$  captures the effect of past innovations or shocks. When the original series is not stationary, ARIMA models introduce differencing. The first difference is defined as

$$\nabla y_t = (1 - B) y_t = y_t - y_{t-1}.$$

An ARIMA  $(p, d, q)$  model is then written as

$$\phi(B) (1 - B)^d y_t = \theta(B) w_t,$$

where  $d$  is the order of regular differencing. This formulation follows the treatment of integrated models for nonstationary data in Shumway and Stoffer [20]. If  $d = 0$ , the model reduces to an ARMA  $(p, q)$ .

Hourly electricity data also display strong seasonal patterns. In particular, observations separated by 24 hours often correspond to the same position in the daily cycle. Seasonal ARIMA models incorporate this structure by adding seasonal autoregressive and moving-average polynomials. Following the multiplicative seasonal ARIMA formulation of Shumway and Stoffer [20], a SARIMA model is denoted by

$$\text{ARIMA}(p, d, q) \times (P, D, Q)_s$$

where  $s$  is the seasonal period. Its general form is

$$\phi(B) \Phi(B^s) (1 - B)^d (1 - B^s)^D y_t = \theta(B) \Theta(B^s) w_t,$$

with

$$\Phi(B^s) = 1 - \Phi_1 B^s - \dots - \Phi_P B^{Ps}, \quad \text{and} \quad \Theta(B^s) = 1 + \Theta_1 B^s + \dots + \Theta_Q B^{Qs}$$

are polynomials in  $B^s$ .

In this thesis, the main seasonal period is set to  $s = 24$ , so that the model can capture daily dependence in hourly data. For example,  $(1 - B^{24}) y_t = y_t - y_{t-24}$  compares the current observation with the same hour of the previous day.

The SARIMAX model extends SARIMA by including external regressors. This can be understood as a regression model with autocorrelated errors, following the framework presented by Shumway and Stoffer [20]. Let

$$\mathbf{x}_t = (x_{1,t}, x_{2,t}, \dots, x_{k,t})^\top$$

be a vector of  $k$  explanatory variables available at time  $t$ . The model is written as

$$y_t = \boldsymbol{\beta}^\top \mathbf{x}_t + u_t,$$

where  $u_t$  is not assumed to be independent noise. Instead,  $u_t$  follows a SARIMA process:

$$\phi(B) \Phi(B^s) (1-B)^d (1-B^s)^D u_t = \theta(B) \Theta(B^s) w_t.$$

Thus, SARIMAX can be interpreted as a regression with exogenous variables and SARIMA errors. This modelling choice is also consistent with the electricity price forecasting literature. Weron [1] describes AR-type models as a central class of statistical models in electricity price forecasting and notes that electricity prices are affected by strong seasonality, spikes and external drivers such as demand and weather conditions. In the same line, models with exogenous inputs are natural in electricity markets because market outcomes are not only driven by their own past values, but also by calendar effects, weather conditions and market fundamentals. In the context of this thesis, the notation is extended to each target  $m$  and bidding zone  $z$ :

$$y_{z,t}^{(m)} = \beta_{z,m}^\top \mathbf{x}_{z,t} + u_{z,t}^{(m)},$$

where  $m \in \{\text{price, purchases, sales}\}$ , and  $z \in \{\text{NORD, CNOR, CSUD, SUD, CALA, SICI, SARD, ITALY}\}$ . The error term is assumed to follow

$$u_{z,t}^{(m)} \sim \text{SARIMA}(p, d, q)(P, D, Q)_{24}.$$

Therefore, the full model can be written as

$$\phi_{z,m}(B) \Phi_{z,m}(B^{24}) (1-B)^d (1-B^{24})^D \left( y_{z,t}^{(m)} - \beta_{z,m}^\top \mathbf{x}_{z,t} \right) = \theta_{z,m}(B) \Theta_{z,m}(B^{24}) w_{z,m,t}.$$

Given observations up to time  $T$ , the  $h$ -step-ahead forecast is obtained as

$$\hat{y}_{T+h} = \hat{\beta}^\top \mathbf{x}_{T+h} + \hat{u}_{T+h}.$$

For the day-ahead setting considered here, forecasts are produced for the next 24 hourly horizons,  $h = 1, \dots, 24$ . This requires constructing the future regressor matrix  $\mathbf{X}_{T+1:T+24}$ .

Model estimation involves the regression coefficients.  $\beta$ , the regular ARMA parameters,  $\phi_i, \theta_j$ , the seasonal parameters,  $\Phi_i, \Theta_j$ , and the innovation variance  $\sigma^2$ . After estimation, residual diagnostics are used to assess whether the remaining errors behave approximately as white noise. Overall, SARIMAX can be understood as the final model in the sequence

$$\text{ARMA} \rightarrow \text{ARIMA} \rightarrow \text{SARIMA} \rightarrow \text{SARIMAX}.$$

ARMA models short-run dependence in stationary series; ARIMA adds ordinary differencing; SARIMA adds seasonal dynamics; and SARIMAX incorporates external regressors while preserving the temporal error structure. This makes SARIMAX a suitable interpretable benchmark for the Italian day-ahead electricity market.

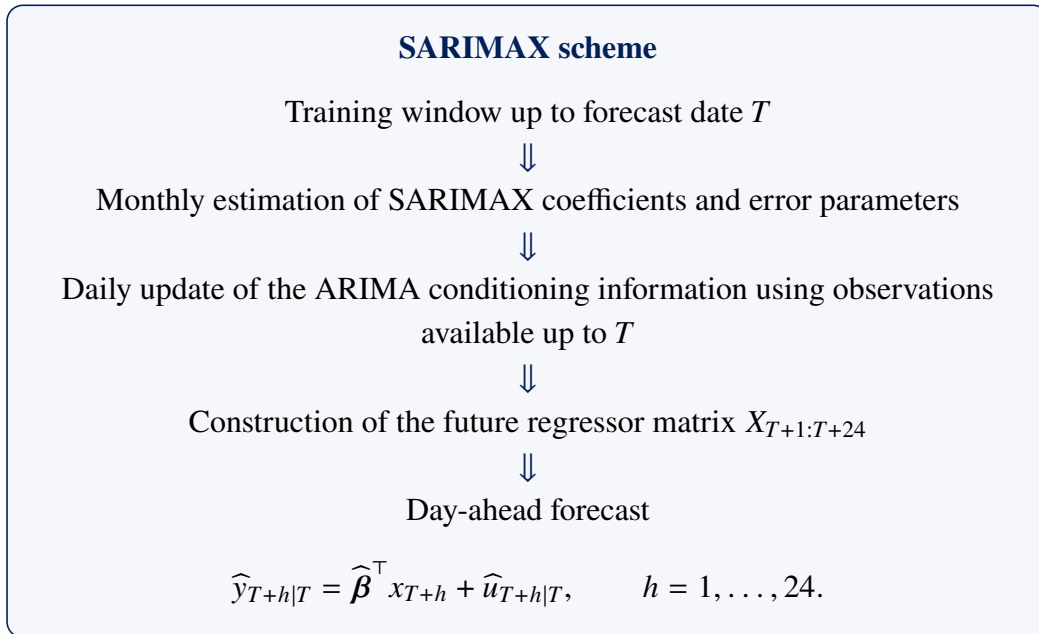


### 3.2.2. Implementation in this study

The SARIMAX model is implemented in R using the `forecast` package [21], [22]. Each model is estimated with `Arima()` and the 24-hour day-ahead forecasts are generated with `forecast()`, using the corresponding matrix of future exogenous regressors for the delivery day. A separate SARIMAX model is fitted for each target variable and physical bidding zone, so that prices, purchases and sales are forecasted independently in each of the seven zones.

The final implementation is deliberately parsimonious. The model uses a seasonal period  $s = 24$ , consistent with the daily structure of the data, and includes a limited set of exogenous regressors: delivery-day weather variables, interpreted as proxies for day-ahead weather forecasts, weekday indicators, and target-specific regional predictors lagged by 24 hours. Own-zone target lags are not included as exogenous regressors, since the local autoregressive dynamics are modelled through the SARIMA error component.

When forecasting a delivery day, the SARIMA error component is forecast recursively in the usual multi-step sense, but this recursion relies on previously forecasted errors rather than on observed delivery-day values. The exogenous component, instead, uses the future regressor matrix  $X_{T+1:T+24}$ , containing the variables available or proxied for each of the 24 delivery hours. The resulting procedure is summarised in Figure 3.1.



**Fig. 3.1.** SARIMAX rolling forecasting scheme.

Two calibration-window lengths are compared during the 2024 validation period: three months and six months. The model is recalibrated monthly, which means that the regression coefficients, SARIMA error parameters and preprocessing of the exogenous regressors are re-estimated at the beginning of each monthly block using only the ob-

servations available at that point. Within the same block, the estimated coefficients and SARIMA parameters are kept fixed. However, before each daily forecast, the fitted model is updated with the observations available up to the current forecast date. This reduces computational cost while still allowing the model to incorporate the most recent observed information.

Several alternatives were explored during validation, including automatic order selection on representative national series, namely PUN for prices and national totals for purchases and sales. However, repeated order selection and frequent full re-estimation were computationally expensive in the complete rolling evaluation. For this reason, the final specification uses fixed low-order SARIMA error structures by target, reported in Table 3.1. The selected calibration window is chosen separately for each target according to validation rMAE against the weekly seasonal naive benchmark, and the resulting specification is then applied to the 2025 test period without further tuning.

| Target    | $p$ | $d$ | $q$ | $P$ | $D$ | $Q$ | $s$ |
|-----------|-----|-----|-----|-----|-----|-----|-----|
| Price     | 1   | 0   | 1   | 1   | 1   | 0   | 24  |
| Purchases | 1   | 0   | 1   | 1   | 0   | 0   | 24  |
| Sales     | 1   | 0   | 1   | 1   | 0   | 0   | 24  |

**TABLE 3.1.** Fixed SARIMA error specifications used in the final SARIMAX implementation.

### 3.3. LEAR

#### 3.3.1. Theoretical background

The LASSO-Estimated AutoRegressive model (LEAR) is a regularized linear forecasting model that combines autoregressive time series regression with automatic variable selection. It can be understood as a parameter rich autoregressive model whose coefficients are estimated using the least absolute shrinkage and selection operator (LASSO). This approach has become a strong statistical benchmark in day-ahead electricity price forecasting, where high-dimensional lag structures and calendar effects are especially relevant [2].

Let  $y_t$  denote the time series to be forecast. A classical autoregressive model represents  $y_t$  as a linear function of its own past values,

$$y_t = \alpha + \sum_{j=1}^p \phi_j y_{t-j} + \varepsilon_t,$$

where  $\varepsilon_t$  is an error term. This formulation captures serial dependence through lagged observations, which is one of the central ideas in time series modelling [20]. In electricity

markets, however, the relevant temporal structure is not restricted to short consecutive lags. Hourly electricity prices and volumes often exhibit strong daily and weekly seasonal patterns, so lagged observations from the previous day or the previous week are typically informative predictors [1], [2].

The autoregressive model can be extended by adding exogenous regressors. In a general ARX-type specification,

$$y_t = \alpha + \sum_{j=1}^p \phi_j y_{t-j} + \mathbf{u}_t^\top \boldsymbol{\gamma} + \varepsilon_t,$$

where  $\mathbf{u}_t \in \mathbb{R}^q$  is a vector of additional predictors. In electricity forecasting, these predictors may include calendar variables, weather variables, lagged market indicators, or other information available at the time the forecast is made. The model remains linear in the parameters, but the number of candidate regressors can become large, especially when several lags and hourly effects are included.

For the regularized formulation used in LEAR, it is convenient to collect both the autoregressive lagged values and the remaining covariates in a single vector of candidate predictors. Let  $\mathcal{L} = \{\ell_1, \dots, \ell_p\}$  denote a pre-specified set of autoregressive lags. Then,

$$\mathbf{x}_t = \left( y_{t-\ell_1}, \dots, y_{t-\ell_p}, u_{1,t}, \dots, u_{q,t} \right)^\top \in \mathbb{R}^{p+q},$$

where the first  $p$  components are lagged values of the target variable and the remaining  $q$  components are exogenous, market-related or calendar predictors. The corresponding coefficient vector is

$$\boldsymbol{\beta} = \left( \beta_{\ell_1}, \dots, \beta_{\ell_p}, \gamma_1, \dots, \gamma_q \right)^\top.$$

The ARX model can then be rewritten compactly as

$$y_t = \beta_0 + \mathbf{x}_t^\top \boldsymbol{\beta} + \varepsilon_t.$$

Estimating a high-dimensional ARX model by ordinary least squares may lead to unstable coefficients and overfitting, particularly when the regressors are highly correlated. This is common in electricity applications, where neighbouring lags, daily lags, weekly lags and calendar effects may contain overlapping information. LEAR addresses this problem by replacing ordinary least squares with LASSO regularization [23]. Given a response vector  $\mathbf{y} = (y_1, \dots, y_T)^\top$  and a design matrix  $\mathbf{X} \in \mathbb{R}^{T \times q}$ , the LASSO estimator is defined as

$$\hat{\boldsymbol{\beta}}_\lambda = \arg \min_{\boldsymbol{\beta}} \left\{ \frac{1}{T} \sum_{t=1}^T (y_t - \beta_0 - \mathbf{x}_t^\top \boldsymbol{\beta})^2 + \lambda \sum_{j=1}^q |\beta_j| \right\},$$

where  $\lambda \geq 0$  is a tuning parameter controlling the strength of the penalty. When  $\lambda = 0$ , the estimator coincides with ordinary least squares. As  $\lambda$  increases, the coefficients are shrunk towards zero and some of them may become exactly equal to zero. Thus, LASSO performs coefficient estimation and variable selection simultaneously.

This sparsity property is the main reason for using LASSO in LEAR models. Instead of manually selecting a small subset of lags and covariates, the modeller can start from a richer set of candidate predictors and let the regularized estimation procedure retain only those that contribute sufficiently to predictive accuracy. In the electricity price forecasting literature, this idea has been shown to be useful for capturing autoregressive intraday structures while avoiding excessively large unrestricted linear models [24]. Lago et al. also identify LEAR as one of the main statistical benchmarks for day-ahead electricity price forecasting, precisely because it combines a rich autoregressive specification with regularized estimation [2].

For hourly day-ahead forecasting, LEAR is usually formulated separately for each delivery hour. Let  $y_{d,h}$  denote the value of the target variable for day  $d$  and hour  $h$ . An hour-specific LEAR specification can be written as

$$y_{d,h} = \beta_{0,h} + \mathbf{x}_{d,h}^\top \boldsymbol{\beta}_h + \varepsilon_{d,h}, \quad h = 1, \dots, 24,$$

where  $\mathbf{x}_{d,h}$  contains the candidate predictors available before delivery. The coefficient vector  $\boldsymbol{\beta}_h$  is allowed to vary with  $h$ , which is important because peak and off-peak hours may have different dynamics and may respond differently to past observations, weather conditions or calendar effects.

Once the coefficients have been estimated, the forecast for day  $d + 1$  and hour  $h$  is given by

$$\hat{y}_{d+1,h} = \hat{\beta}_{0,h} + \mathbf{x}_{d+1,h}^\top \hat{\boldsymbol{\beta}}_{h,\lambda}.$$

An important thing to take into account before applying the LASSO penalty is to standardize the predictors. This is necessary because the penalty acts directly on the magnitude of the coefficients, and variables measured on different scales would otherwise be penalized unequally. For a predictor  $x_j$ , the standardized version is

$$x_{j,t}^* = \frac{x_{j,t} - \bar{x}_j}{s_j},$$

where  $\bar{x}_j$  and  $s_j$  are the sample mean and standard deviation computed on the training sample. In rolling-window forecasting, these quantities must be computed using only past observations, so that the transformation does not introduce look-ahead bias.

The tuning parameter  $\lambda$  is selected through a validation procedure. Conceptually, one chooses

$$\hat{\lambda} = \arg \min_{\lambda \in \Lambda} \mathcal{L}_{\text{val}}(\lambda),$$

where  $(\mathcal{L}_{\text{val}})$  is a forecasting loss computed on a validation sample. In time series applications, the validation scheme must preserve the temporal order of the data. Random cross-validation is generally inappropriate when it mixes past and future observations.

LEAR therefore occupies an intermediate position between classical statistical forecasting and machine learning. It preserves the transparency of a linear autoregressive

regression model, while regularization allows the use of a large set of candidate predictors. Its coefficients should nevertheless be interpreted carefully. A non-zero coefficient indicates predictive usefulness within the selected specification and sample. Consequently, the adequacy of LEAR is mainly assessed through out-of-sample forecasting accuracy, comparison against benchmark models and the stability of the selected predictors [2], [23].

### 3.3.2. Implementation in this study

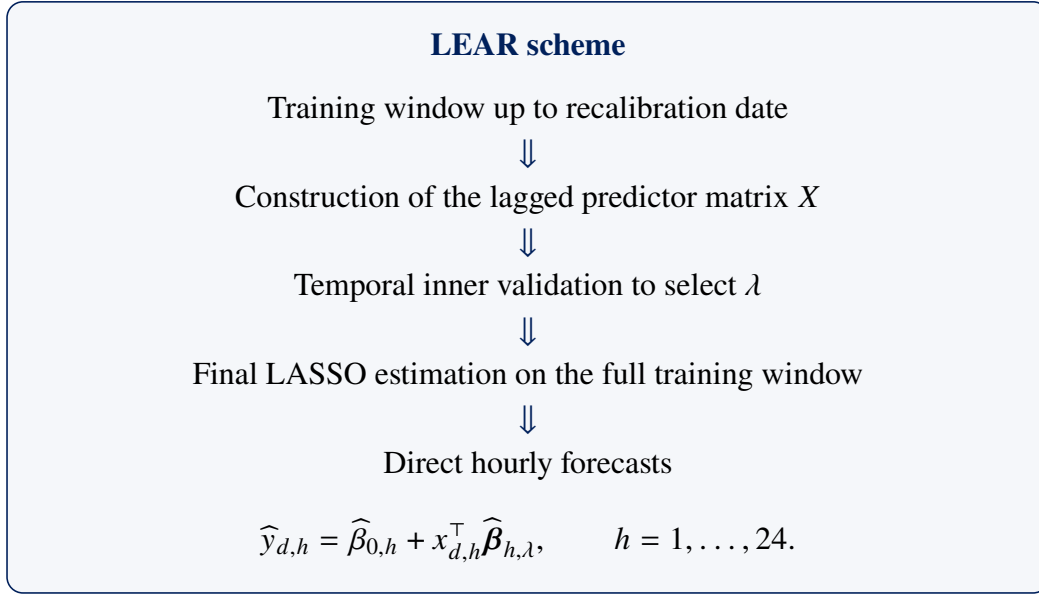
The LEAR model is implemented in R using the `glmnet` package [25]. Following the hour-specific formulation introduced in Section 3.3.1, a separate LASSO regression is estimated for each target, bidding zone and delivery hour. Thus, for each target-zone pair, the 24 hourly forecasts are obtained from 24 hour-specific equations rather than from a single multi-output model.

LEAR introduces temporal dependence directly through the predictor matrix. The candidate predictors include own-target lags at 24, 48 and 168 hours, selected lagged market variables, national aggregate variables, calendar indicators, weather variables and lagged cross-zone market information. Cross-zone variables are included only through lagged values, so that each zonal equation can use information from the remaining Italian physical zones without introducing contemporaneous information from the delivery day.

Four monthly recalibration strategies are compared during the 2024 validation period: rolling windows of 6, 12 and 24 months, and an expanding window using all past observations. The best strategy is selected separately for each target according to validation rMAE against the weekly seasonal naive benchmark. The selected strategies are then applied to the 2025 test period without further tuning. Within each monthly block, the fitted LEAR coefficients remain fixed; day-to-day adaptation enters through the updated lagged predictors used to construct the forecast rows.

The regularization parameter  $\lambda$  is selected at each recalibration by a temporal holdout procedure. The last 30 days of the current training window are used as an inner validation sample, while the previous observations form the inner training sample. A grid of candidate  $\lambda$  values is fitted on the inner training period and evaluated by MAE on the inner validation period. The value with the lowest validation error is then used to refit the LASSO model on the full training window.

The preprocessing is also performed separately within each recalibration window. Numerical predictors are processed using training-window statistics only, and categorical variables are converted into dummy variables. The resulting design matrix is passed to `glmnet`, which standardises the predictors internally before applying the LASSO penalty. This ensures that the regularization is applied on a comparable scale and that no information outside the current training window is used when constructing the model. The whole LEAR implementation procedure is summarised in Figure 3.2.



**Fig. 3.2.** LEAR scheme based on the implemented forecasting pipeline.

### 3.4. Random Forest

#### 3.4.1. Theoretical background

Random Forest is an ensemble learning method introduced by [13]. It builds a collection of decision trees and combines their predictions in order to reduce the instability of individual trees. In regression, each tree produces a numerical prediction and the forest prediction is obtained by averaging the predictions across trees. This method is a bagging-type procedure with an additional source of randomness added. At each split only a random subset of predictors is considered as candidate splitting variables [13].

Let  $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$  be a supervised regression sample, where  $\mathbf{x}_i \in \mathbb{R}^p$  is a vector of predictors and  $y_i \in \mathbb{R}$  is the response. A regression tree recursively partitions the predictor space into terminal regions  $R_1, \dots, R_M$ . For a new observation  $\mathbf{x}$ , the prediction is given by the average response of the training observations falling in the same terminal region:

$$\hat{f}_{\text{tree}}(\mathbf{x}) = \sum_{m=1}^M \bar{y}_{R_m} \mathbf{1}\{\mathbf{x} \in R_m\}, \quad \bar{y}_{R_m} = \frac{1}{|R_m|} \sum_{i: \mathbf{x}_i \in R_m} y_i.$$

The splits are chosen recursively to reduce within-node prediction error, typically measured by squared error in regression problems.

A Random Forest grows  $B$  randomized trees. Each tree is fitted using a bootstrap sample of the training data and a random subset of candidate predictors is considered at each split. If  $T_b(\mathbf{x})$  denotes the prediction of the  $b$ -th tree, the Random Forest predictor is

$$\hat{f}_{RF}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}).$$

Averaging reduces the variance of individual trees, while the randomization in the construction of the trees aims to reduce their correlation. As discussed by [13], the accuracy of regression forests depends on both the predictive strength of the individual trees and the correlation among their errors.

The method can also be extended to a multi-output regression setting, where the response associated with each observation is a vector rather than a scalar. Multivariate extensions of Random Forest are discussed by [26], who consider the case in which several related response variables are modelled jointly. In this setting,

$$\mathbf{y}_i = (y_{i1}, y_{i2}, \dots, y_{iK})^\top \in \mathbb{R}^K,$$

and the model estimates a function  $f : \mathbb{R}^p \rightarrow \mathbb{R}^K$ .

The output vector may represent several related quantities to be forecast jointly, such as different horizons, locations or market variables. The exact definition of this vector is part of the empirical specification of the model. In a multi-output regression tree, terminal-node predictions are vector-valued averages:

$$\bar{\mathbf{y}}_{R_m} = \frac{1}{|R_m|} \sum_{i: \mathbf{x}_i \in R_m} \mathbf{y}_i.$$

Under a squared-error formulation, candidate splits are evaluated by the total within-node dispersion of the multivariate responses:

$$(j^*, s^*) = \arg \min_{j, s} \left[ \sum_{i \in R_L(j, s)} \|\mathbf{y}_i - \bar{\mathbf{y}}_L\|_2^2 + \sum_{i \in R_R(j, s)} \|\mathbf{y}_i - \bar{\mathbf{y}}_R\|_2^2 \right].$$

This implies that the tree structure is learned jointly for all components of the response vector. Consequently, a multi-output Random Forest can exploit common patterns across related outputs [26]. However, Random Forest remains a supervised learning model applied to time-indexed data, rather than a classical time series model with an explicit autoregressive error structure. Temporal dependence must therefore be introduced through the construction of the input variables, such as autoregressive lags, seasonal lags, calendar indicators and exogenous predictors. In the empirical analysis, the model is estimated using only information available before the forecast origin and evaluated under the same rolling forecasting framework, benchmarks and error metrics as the remaining models, following the recommendations for rigorous electricity price forecasting comparisons discussed by [2].

### 3.4.2. Implementation in this study

The Random Forest models are implemented in Python using the `scikit-learn` library [27]. As discussed in the previous section (Section 3.4.1), the model is applied as

a supervised nonlinear forecasting method, the temporal dependence is modelled through the construction of lagged predictors, calendar variables and exogenous regressors.

The implementation considers several multi-output structures, which differ in the definition of the response vector and therefore in the type of joint dependence that the model is allowed to exploit. This choice is treated as an empirical modelling decision and is evaluated during the 2024 validation period. The candidate structures are summarised in Table 3.2.

| Structure        | Output vector        | Main implication                                                                               |
|------------------|----------------------|------------------------------------------------------------------------------------------------|
| rf_H_24h         | 24 hourly values     | Learns a joint tree structure for the full daily profile of one target and one zone.           |
| rf_Z_7zones      | 7 zonal values       | Shares the tree structure across the seven physical bidding zones for a given target and hour. |
| rf_HZ_24h_7zones | $24 \times 7$ values | Predicts the complete day-ahead zonal profile of a given target within one multi-output model. |
| rf_T_3targets    | 3 target values      | Shares the tree structure across prices, purchases and sales for a given zone and hour.        |

**TABLE 3.2.** Candidate multi-output structures considered for Random Forest.

The same ideas for the predictors variables described for the previous models are imposed here. In particular, market variables enter the Random Forest only through lagged values, while delivery-day weather variables are used as proxies for day-ahead weather forecasts. The distinctive element of this implementation is not the information set itself, but the way in which the supervised learning problem is organised under different multi-output structures. Depending on the structure, the response vector may collect hourly values, zonal values, or different market targets, and the input matrix is reshaped accordingly.

The Random Forest is recalibrated monthly using a rolling training window of 24 months. At each recalibration date, the model is fitted using only observations prior to the forecasted block. The validation period is used to compare the candidate output structures, while the final test period is used only for the out-of-sample comparison of the selected specification.

The main hyperparameters are kept fixed across targets and zones in order to avoid an excessively large validation problem. In the full configuration, each forest uses  $B = 300$  trees, bootstrap sampling, squared-error splitting and `min_samples_leaf` equal to 5, among others parameters. These settings provide a compromise between flexibility, variance control and computational feasibility in the rolling evaluation.



### 3.5. XGBoost

#### 3.5.1. Theoretical background

Extreme Gradient Boosting, commonly known as XGBoost, is a scalable implementation of gradient tree boosting introduced by [14]. This model belongs to the family of ensemble methods and constructs the final predictor as an additive combination of regression trees. Given a set of input variables  $x_i$ , the prediction for observation  $i$  can be written as

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i),$$

where each  $f_k$  is a regression tree and  $K$  is the total number of boosting iterations. Unlike Random Forest, where trees are mainly grown independently and then averaged (see Section 3.4.1), boosting methods build trees sequentially. Each new tree is fitted to improve the current ensemble by correcting the remaining prediction errors.

Following [14], at iteration  $t$ , XGBoost adds a new tree  $f_t$  by minimizing a regularized objective function of the form

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t),$$

where  $l(\cdot)$  is the loss function and  $\Omega(f_t)$  penalizes the complexity of the tree. A common regularization term is

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2,$$

where  $T$  is the number of leaves,  $w_j$  is the score assigned to leaf  $j$ , and  $\gamma$  and  $\lambda$  control the degree of regularization.

In addition, XGBoost incorporates shrinkage and column subsampling as mechanisms to reduce overfitting and improve generalization [14].

In summary, XGBoost provides a regularized boosting alternative to the bagging approach used by Random Forest. Both methods are based on regression trees, but they differ in the way the ensemble is constructed: Random Forest averages randomized trees grown in parallel, whereas XGBoost adds trees sequentially to improve the current predictor. This makes XGBoost particularly relevant as a nonlinear machine learning benchmark, since it combines the flexibility of tree-based models with explicit regularization and boosting-based optimization.

#### 3.5.2. Implementation in this study

The XGBoost model is implemented in Python using the `xgboost` package and its `XGBRegressor` interface [14]. In contrast to the multi-output Random Forest, XGBoost

is implemented in a long hourly format, where each observation corresponds to one target-zone-hour combination, and a separate model is estimated for each target and physical bidding zone. The 24 day-ahead forecasts for a delivery day are then obtained by applying the same fitted target-zone model to the 24 hourly predictor rows of that day.

The information set follows the same leakage-safe restrictions as in the previous specifications. Market information is introduced through lagged variables, delivery-day weather variables are treated as proxies for day-ahead weather forecasts, and cross-zone predictors include lagged information from the remaining physical zones only.

This model is also recalibrated monthly. During the 2024 validation period, three training-window strategies are compared: a 12-month rolling window, a 24-month rolling window and an expanding window starting in 2021. For each target, the combination of training window and hyperparameter configuration with the lowest validation rMAE against the weekly seasonal naive benchmark is selected. The selected target-specific strategies are then used in the 2025 test period without further tuning.

The hyperparameter search is kept deliberately compact because the model is re-estimated repeatedly in a rolling evaluation. The candidate configurations differ in tree depth, learning rate, maximum number of boosting rounds, sampling rates and regularization strength.

Within each recalibration window, the last 30 days are used as an inner validation sample for early stopping, while the previous observations are used to fit the boosting trees. Early stopping is based on MAE on this inner validation sample and stops the boosting process if the validation error does not improve for 50 consecutive rounds. This mechanism determines the effective number of boosting iterations within each recalibration block and helps control overfitting.

## 3.6. LSTM

### 3.6.1. Theoretical background

Deep learning extends the supervised learning framework by allowing the model to learn intermediate representations of the input data [28]. Recurrent neural networks (RNNs) are neural network architectures designed for sequential data. Let

$$x_1, x_2, \dots, x_T,$$

denote an input sequence, where  $x_t \in \mathbb{R}^p$  contains the variables observed at time  $t$ . A simple recurrent network updates an internal hidden state according to

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b_h),$$

where  $h_t \in \mathbb{R}^m$  is the hidden state,  $W_x$ ,  $W_h$  and  $b_h$  are trainable parameters, and  $\phi$  is a nonlinear activation function. The hidden state acts as a learned summary of the

information processed up to time  $t$ . Therefore, unlike feed-forward neural networks applied to independent input vectors, RNNs explicitly account for the order of the observations by recursively updating their internal representation.

However, standard recurrent networks are difficult to train when relevant information must be propagated over long time intervals. In particular, error signals propagated backwards through time may decay or grow exponentially, leading to the well-known vanishing and exploding gradient problems. This makes it hard for a standard RNN to learn long-range dependencies in a stable way [15]. Long Short-Term Memory (LSTM) networks were introduced by [15] to address this limitation through a more structured memory mechanism.

The central idea of an LSTM is to separate the hidden representation from a memory state. At each time step, the model keeps both a hidden state  $h_t$  and a cell state  $c_t$ . The cell state acts as an internal memory, while a set of gates controls how information is stored, forgotten and exposed. A commonly used formulation of an LSTM cell is

$$\begin{aligned} f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), \\ h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

Here,  $\sigma(\cdot)$  denotes the logistic sigmoid function,  $\tanh(\cdot)$  is the hyperbolic tangent function, and  $\odot$  denotes element-wise multiplication. The matrices  $W$ . and  $U$ ., together with the bias vectors  $b$ ., are learned from the data.

The forget gate  $f_t$  determines how much of the previous memory  $c_{t-1}$  is retained. The input gate  $i_t$  controls how much new candidate information  $\tilde{c}_t$  is written into the memory. The output gate  $o_t$  controls how much of the internal memory is used to form the hidden state  $h_t$ . Since the gates are based on sigmoid functions, their components take values between zero and one and can be interpreted as soft switches. The candidate memory uses a hyperbolic tangent transformation, which produces bounded positive or negative values and therefore allows the model to update the memory in both directions.

In the context of day-ahead electricity market forecasting, the LSTM is used as a supervised sequential model. For each forecast origin, the input can be represented as a historical window

$$X_i = (x_{i-L+1}, \dots, x_i) \in \mathbb{R}^{L \times p},$$

where  $L$  is the length of the look-back window and  $p$  is the number of input variables.

The LSTM processes the window in chronological order and updates its hidden state at each step:

$$h_{i,1}, h_{i,2}, \dots, h_{i,L}.$$

The last hidden state  $h_{i,L}$  is interpreted as a learned summary of the information contained in the historical window. This summary is then passed to an output layer, which transforms it into the vector of 24 hourly forecasts for the next delivery day:

$$\widehat{\mathbf{y}}_i = (\widehat{y}_{i,1}, \dots, \widehat{y}_{i,24}) = W_y h_{i,L} + b_y.$$

Here,  $W_y \in \mathbb{R}^{24 \times m}$ ,  $b_y \in \mathbb{R}^{24}$ , and  $m$  denotes the dimension of the LSTM hidden state. Thus, the model follows a direct multi-output forecasting strategy, in the sense that the full day-ahead hourly profile is generated at once, rather than recursively producing one forecast and then using it as an input for the next horizon.

### 3.6.2. Implementation in this study

The LSTM models are implemented in Python using PyTorch [29]. Preprocessing steps such as scaling and categorical encoding are implemented with scikit-learn utilities [27]. The main LSTM specification follows the direct multi-output strategy described in the Subsection 3.6.1, for each target and physical bidding zone, the model outputs the 24 hourly forecasts for the next delivery day.

The historical sequence includes the same information blocks used in the previous machine learning models, but arranged as an ordered sequence rather than as a tabular lag matrix. These blocks include own-zone market variables, common national market variables, calendar variables and cross-zone information from the remaining physical bidding zones. The important characteristic is the sequential representation, the LSTM processes the previous  $L$  hourly observations in chronological order and learns its own internal summary of the recent past.

The main validation grid considers two look-back lengths,  $L = 168$  and  $L = 336$  hours, corresponding to one and two weeks of hourly history. It also varies the hidden-state dimension, the number of LSTM layers, the learning rate and the batch size. Dropout is set to zero for one-layer LSTMs and to 0.2 for two-layer LSTMs. The maximum number of epochs is set to 50, but training is stopped earlier when the validation loss does not improve. The candidate strategies are evaluated during the 2024 validation period, and the best specification is selected separately for each target according to validation rMAE against the weekly seasonal naive benchmark.

Training is performed using the Adam optimizer and mean squared error loss on standardized targets. Within each training window, the last 30 days are used as an inner validation sample for early stopping. All preprocessing transformations and target standardization are fitted using only the corresponding training sample and are then applied to the validation or test samples. Predictions are transformed back to the original scale before evaluation.

The main LSTM specification is first estimated using an expanding training window, since neural networks usually benefits from a large number of observations. However,

in this application, using all past observations is not necessarily optimal. As shown in the exploratory analysis in Section 2.3, the sample contains strong changes in market conditions, most notably during the 2022 energy crisis. Therefore, older observations may increase the amount of training data but may also expose the model to regimes that are less representative of the final forecasting period.

For this reason, an additional rolling-window LSTM variant is considered. This variant keeps the architectures selected in the main expanding-window validation but changes the training design to a 24-month rolling window. The purpose is to assess whether restricting the estimation sample to more recent observations provides a more stable specification for the final forecasting period.

As a final extension, a recursive LSTM variant is also implemented. The motivation is to compare the direct multi-output strategy with a simpler one-step-ahead formulation. This specification is treated as a complementary experiment, since the recursive setup also requires a more restrictive leakage-safe input design, which consist in, within the delivery day, only variables that can be updated recursively or treated as future-known covariates can be used. In the direct model, the network must learn the full 24-hour delivery profile at once. However, this joint formulation also imposes a common model structure across the forecasted horizons, which may reduce flexibility when the different horizons exhibit heterogeneous dynamics [30]. In this particular application, this issue is relevant, as it has been shown in the exploratory analysis in Section 2.3 a high 24-hour profile variations in the data. Of course, this implementation has a drawback, as forecast errors can accumulate across the recursive path, since predictions produced for earlier delivery hours are used as inputs for later horizons.

For computational reasons, the recursive LSTM is treated as a complementary specification rather than as the main deep learning design.

Table 3.3 summarises the three LSTM implementations considered in the empirical analysis.

| Specification              | Training window         | Output construction                   |
|----------------------------|-------------------------|---------------------------------------|
| Direct LSTM                | Expanding window        | Direct 24-hour multi-output forecast. |
| Rolling-24m direct LSTM    | 24-month rolling window | Direct 24-hour multi-output forecast. |
| Recursive rolling-24m LSTM | 24-month rolling window | Recursive forecast.                   |

**TABLE 3.3.** Summary of the LSTM specifications considered.

### 3.7. Temporal Convolutional Network

#### 3.7.1. Theoretical background

Temporal Convolutional Networks (TCNs) are convolutional neural networks designed for sequential data. In forecasting applications, they learn temporal representations from a fixed historical window by applying one-dimensional convolutions along the time axis. The architecture considered in this thesis follows the generic TCN formulation of [16], based on causal convolutions, dilated convolutions and residual blocks.

Let

$$X_i = (x_{i,1}, x_{i,2}, \dots, x_{i,L}) \in \mathbb{R}^{L \times p}$$

denote the input sequence associated with forecast origin  $i$ , where  $L$  is the look-back window length and  $p$  is the number of variables observed at each time step. In this thesis, these variables may include lagged market quantities, calendar indicators, weather-related predictors and cross-zone information. The model is used to learn a nonlinear mapping from the historical window to the full day-ahead hourly forecast vector,

$$\hat{\mathbf{y}}_i = (\hat{y}_{i,1}, \dots, \hat{y}_{i,24}).$$

The basic operation in a TCN is a one-dimensional convolution over time. Since the model is used for forecasting, the convolution must be causal, this means that the representation at a given time can only depend on current and past observations, not on future ones. Bai et al. [16] define the dilated convolution operation as

$$F(s) = (x *_d f)(s) = \sum_{i=0}^{k-1} f(i) \cdot x_{s-d \cdot i},$$

where  $x$  is the input sequence,  $f$  is the convolutional filter,  $k$  is the filter size,  $d$  is the dilation factor and  $s$  denotes the position in the sequence. When  $d = 1$ , this corresponds to a standard causal convolution. When  $d > 1$ , the filter skips observations and can therefore cover a longer portion of the past.

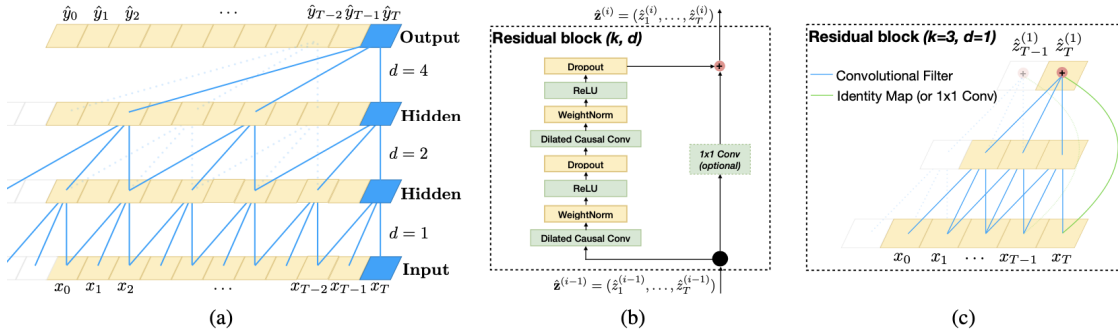
This dilation mechanism is important, by stacking convolutional layers with increasing dilation factors, the TCN can enlarge its receptive field, that is, the amount of past information that can affect the output representation. In the hourly setting of this thesis, this is particularly relevant because the architecture should be able to cover at least the weekly lag of 168 hours.

In practice, the TCN is built from residual blocks. Each block applies a nonlinear transformation  $F(\cdot)$ , composed of causal dilated convolutions, nonlinear activations, normalization and dropout. The output of the block is then combined with its input through a residual connection. Bai et al. [16] write this structure as

$$o = \text{Activation}(x + F(x)).$$

If the input and output of the block have different dimensions, a  $1 \times 1$  convolution is used to match the number of channels before the residual addition. The purpose of this structure is to let each block to learn modifications to the identity mapping rather than the entire transformation, which has been shown to benefit very deep networks [16]. This helps information flow through deeper stacks of convolutional layers and makes the optimization more stable.

Figure 3.3 summarizes the main elements of the architecture: causal dilated convolutions, residual blocks, dropout and the residual connection.



**Fig. 3.3.** Main components of the Temporal Convolutional Network architecture. Panel (a) shows a stack of causal dilated convolutional layers with increasing dilation factors. Panel (b) shows the structure of a residual block, including dilated causal convolutions, weight normalization, ReLU activations, dropout and the optional  $1 \times 1$  convolution used to match dimensions. Panel (c) illustrates the residual connection inside a block. Source: [16].

After the final residual block, the hidden representation at the last position of the input window is used as a summary of the available historical information. A dense output layer then maps this representation into the 24 hourly forecasts for the next delivery day.

The main advantage of TCNs is that they can learn nonlinear temporal patterns over multiple time scales while preserving the causal structure required in forecasting. Dilated convolutions allow the model to access long histories efficiently, and the convolutional structure avoids the need to process the sequence recursively. At the same time, TCNs require careful architectural choices, especially regarding the window length, dilation structure, receptive field and dropout rate.

### 3.7.2. Implementation in this study

The TCN model is also implemented in Python using PyTorch [29]. Preprocessing steps such as scaling and categorical encoding are implemented with scikit-learn utilities [27]. The implementation follows the architecture described in the theoretical background in Section 3.7.1, the model processes a historical hourly sequence, and a dense output head produces the day-ahead forecast. As a result, the main TCN specification is



formulated as a direct 24-hour multi-output model. The model is estimated separately for each target-zone pair.

The TCN encoder is built from residual blocks with causal dilated one-dimensional convolutions, ReLU activations, dropout and weight normalization. Causality is enforced by explicit left padding, so that the representation at each time step depends only on current and past observations. Dilations increase by powers of two across residual blocks.

The direct TCN receives two types of inputs. The first input is the historical hourly sequence, which contains the information available before the delivery day and is processed by the causal dilated convolutional encoder. This branch is used to extract a temporal representation of recent market dynamics. The second input contains delivery-day covariates that are known or proxied at the forecast origin, mainly calendar variables and weather forecasts/proxies. These variables are passed through a connected layer and then combined with the temporal representation extracted by the TCN encoder. The final output head uses this combined representation to produce the 24 hourly forecasts.

A compact validation grid is used to compare architectures with different look-back lengths, convolutional filter sizes, channel widths, dropout rates and learning rates. The look-back window takes values of 168 and 336 hours, corresponding to one and two weeks of hourly history. The remaining hyperparameters are kept within a limited range in order to compare moderate changes in capacity and regularization while keeping the rolling evaluation computationally feasible. Candidate architectures are constructed so that the resulting receptive field covers at least 168 hours, ensuring that weekly temporal dependencies can in principle be represented by the convolutional stack.

The training protocol is kept consistent with the LSTM implementation (Section 3.6.2). Network parameters are estimated using the Adam optimizer and mean squared error loss on standardized targets. The last 30 days of the training block are not used to update the network weights, but only to monitor validation loss for early stopping. Gradient clipping and  $L_2$  weight decay are used to stabilize training. As with the LSTM models, monthly recalibration is not performed because of the computational cost of repeatedly training deep neural networks.

In addition to the main expanding-window specification, a 24-month rolling-window TCN is considered. This mirrors the rolling-window LSTM variant and is motivated by the same issue discussed in Section 3.6.2: although deep learning models often benefit from larger samples, the exploratory analysis in Section 2.3 shows strong changes in market conditions during the sample period. The rolling-window variant therefore tests whether restricting the training sample to more recent observations improves robustness. No further hyperparameter search is introduced for this specification.

Finally, a recursive rolling-24m TCN is implemented as a complementary forecasting strategy, again in parallel with the recursive LSTM. The direct TCN predicts the full daily profile jointly, which is coherent with the day-ahead task, but imposes a common model structure across all horizons. The recursive TCN instead learns a one-hour-ahead



forecasting problem and generates the 24-hour profile sequentially. This may make the local prediction task simpler, although it also introduces the usual risk of error propagation across horizons. Table 3.4 summarises the three TCN specifications considered in the empirical analysis.

| Specification          |             | Training window         | Output construction                           |
|------------------------|-------------|-------------------------|-----------------------------------------------|
| Direct TCN             |             | Expanding window        | Direct 24-hour multi-output forecast.         |
| Rolling-24m direct TCN |             | 24-month rolling window | Direct 24-hour multi-output forecast.         |
| Recursive TCN          | rolling-24m | 24-month rolling window | The 24-hour profile is generated sequentially |

**TABLE 3.4.** Summary of the TCN specifications considered.

## 4. Forecast results

### 4.1. SARIMAX

The SARIMAX model was included as a statistical benchmark with explicit temporal dependence, it is explained in Section 3.2. The validation stage selected the rolling calibration window separately by target. As shown in Table 4.1, the 6-month window was selected for prices, while the 3-month window was selected for purchases and sales.

| Target    | Selected strategy | Window   | MAE   | rMAE  |
|-----------|-------------------|----------|-------|-------|
| Price     | sarimax_6m        | 6 months | 12.8  | 0.811 |
| Purchases | sarimax_3m        | 3 months | 338.0 | 1.240 |
| Sales     | sarimax_3m        | 3 months | 527.0 | 0.810 |

**TABLE 4.1.** SARIMAX strategy selected in validation by target.

The final test results are reported in Table 4.2. SARIMAX improves on the weekly naive benchmark for prices and sales, with average rMAE values of 0.817 and 0.816, respectively. This corresponds to an average MAE reduction of approximately 18% in both cases. The result is consistent across all price zones, where rMAE ranges from 0.791 in SUD to 0.852 in CNOR and CSUD. For sales, the improvement is also clear in most zones, especially SUD, SARD and SICI. The main exception is NORD, where the model is essentially tied with, and slightly worse than, the weekly naive benchmark. The complete test performance by model, target and zone is reported in the Table A.4 in the Appendix.

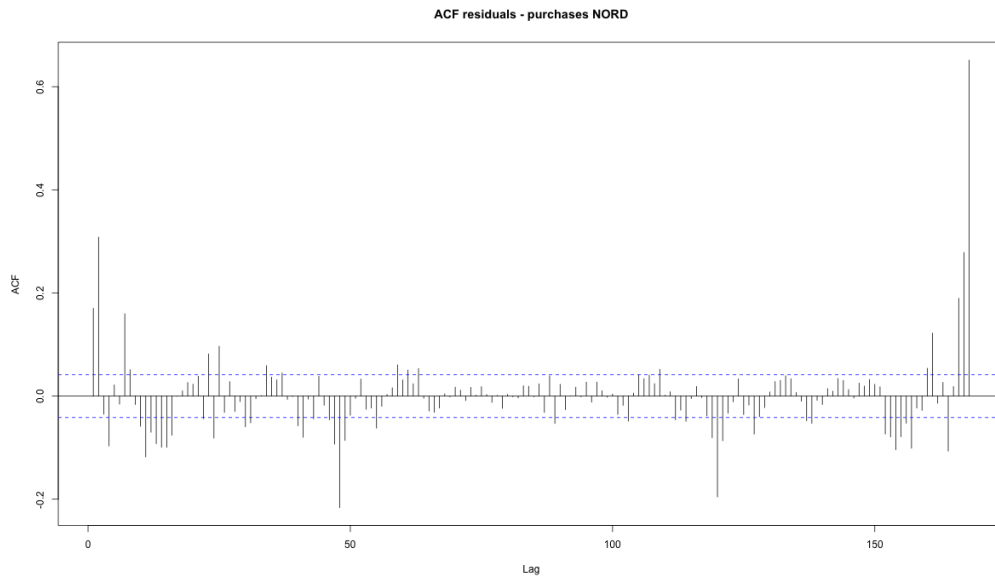
| Target    | MAE   | RMSE  | NMAE  | NRMSE | rMAE  |
|-----------|-------|-------|-------|-------|-------|
| Price     | 12.8  | 20.3  | –     | –     | 0.817 |
| Purchases | 333.0 | 528.0 | 0.062 | 0.095 | 1.160 |
| Sales     | 526.0 | 716.0 | 0.158 | 0.208 | 0.816 |

**TABLE 4.2.** SARIMAX performance on the 2025 test set, averaged across zones.

The weakest SARIMAX results are obtained for purchases. In this target, the average rMAE is 1.160, so the model is worse than the weekly naive benchmark on average. The deterioration is especially visible in CNOR, NORD and CSUD, with rMAE values of 1.63, 1.33 and 1.26, respectively. This suggests that the purchase series are not well captured by the restricted SARIMAX specification used here.

This is consistent with the strong weekly persistence observed in purchase volumes. In CNOR, NORD and CSUD, the correlation with the 168-hour lag is above 0.88 (see Table A.1 in the Appendix). This suggests that the purchase series contain a strong zone-specific weekly pattern that is directly exploited by the weekly naive benchmark. The SARIMAX specification used here includes a daily seasonal error component and weekday effects, but it does not include the own-zone 168-hour lag as an explicit predictor. As a result, the additional model complexity does not compensate for the loss of this simple but highly informative weekly reference.

The residual diagnostics support this interpretation. Figure 4.1 shows a representative residual autocorrelation function. The residuals still display visible dependence at seasonal lags, especially around the weekly frequency. The residuals still display visible dependence at seasonal lags, particularly around the daily and weekly frequencies. The Ljung–Box tests at lags 24 and 168 reject the null hypothesis of no residual autocorrelation in all diagnostic cases considered, with p-values below 0.001. Therefore, the fitted SARIMAX models do not fully remove the temporal structure of the series. This indicates that some systematic dependence remains unexplained.



**Fig. 4.1.** Autocorrelation function of SARIMAX residuals for purchases in NORD.

Overall, SARIMAX provides a useful statistical reference for the forecasting comparison. It performs reasonably well for prices and sales, but it is not a competitive alternative for purchases in this empirical setting. For this target, the weekly naive benchmark performs better on average and is also much simpler computationally.

## 4.2. LEAR

The LEAR was included as a regularized autoregressive model, as explained in Section 3.3. The candidate feature set includes own-target lags, local market lags, lagged cross-zone variables, national market aggregates, calendar variables and weather variables. The LASSO penalty then selects a sparse subset of the expanded design matrix at each recalibration.

The validation stage selected the LEAR specification separately by target. As shown in Table 4.3, the expanding window was selected for prices and sales, whereas the 24-month rolling window was selected for purchases.

| Target    | Window type | Window length | Validation rMAE |
|-----------|-------------|---------------|-----------------|
| Price     | Expanding   | –             | 0.738           |
| Purchases | Rolling     | 24 months     | 0.581           |
| Sales     | Expanding   | –             | 0.650           |

**TABLE 4.3.** LEAR strategy selected in validation by target.

On the 2025 test set, LEAR clearly improves on the weekly naive benchmark for all three targets, as it is shown in Table 4.4. The values obtained represent a relative MAE reductions of approximately 28%, 39% and 34% with respect to the weekly naive benchmark. This improvement is also robust across zones. This is particularly relevant for purchases, where SARIMAX was not competitive against the weekly naive benchmark. LEAR avoids this limitation by including explicit autoregressive lags, including the 168-hour lag, while controlling the enlarged predictor set through regularization.

| Target    | MAE   | RMSE  | NMAE  | NRMSE | rMAE  |
|-----------|-------|-------|-------|-------|-------|
| Price     | 11.3  | 17.4  | –     | –     | 0.716 |
| Purchases | 157.0 | 262.0 | 0.033 | 0.055 | 0.608 |
| Sales     | 395.0 | 512.0 | 0.130 | 0.168 | 0.658 |

**TABLE 4.4.** LEAR performance on the 2025 test set, averaged across zones.

The sparsity induced by the LASSO penalty is summarized in Table 4.5. The column *Mean features* refers to the number of columns in the final design matrix after preprocessing. This is affected in particular by the transformation of the categorical predictors into dummy variables. It varies slightly because the number of dummies generated by the lagged MTI variable varies slightly across estimation windows, depending on the market-technology categories observed in the corresponding training sample. The fitted models retain only a subset of these columns. On average, price equations use around 22 non-zero coefficients, whereas purchases and sales use around 38 and 42, respectively.

These results suggest that the volume targets require a richer set of predictors, while the price equations are more parsimonious.

| Target    | Mean features | Mean non-zero | Median | Q25 | Q75 |
|-----------|---------------|---------------|--------|-----|-----|
| Price     | 93.3          | 22.3          | 15     | 10  | 27  |
| Purchases | 91.4          | 38.0          | 34     | 20  | 54  |
| Sales     | 93.3          | 41.6          | 36     | 15  | 70  |

**TABLE 4.5.** Distribution of non-zero LEAR coefficients on the 2025 test set.

These results show that LEAR provides a strong compromise between interpretability and predictive accuracy. The model remains linear in the predictors, allowing the use of explicit lagged variables, cross-zone information and national market conditions but controlling the dimension of the problem.

### 4.3. Random Forest

The Random Forest model was implemented as a non-linear machine learning model with alternative multi-output structures, as described in Section 3.4. The validation stage selected both the output structure and the rolling-window length separately by target. As shown in Table 4.6, the 7-zone output structure was selected for prices, whereas the 24-hour output structure was selected for purchases and sales. In all cases, the 24-month rolling window was preferred.

| Target    | Selected output structure | Window    | Validation rMAE |
|-----------|---------------------------|-----------|-----------------|
| Price     | 7-zone output             | 24 months | 0.712           |
| Purchases | 24-hour output            | 24 months | 0.639           |
| Sales     | 24-hour output            | 24 months | 0.674           |

**TABLE 4.6.** Random Forest structure selected in validation by target.

The target-specific selection can be interpreted from the way multi-output Random Forests learn the tree structure. As discussed in Section 3.4.1, each split is selected by reducing the joint squared-error dispersion of all components of the response vector. Therefore, a multi-output structure is most useful when the outputs that are modelled jointly have comparable scales and share similar predictive patterns.

This explains why the 7-zone output structure is selected for prices. In validation, this specification obtains an rMAE of 0.712, compared with 0.769 for the 24-hour output structure. Zonal prices are measured in the same unit, have similar magnitudes across the Italian bidding zones and are affected by common market conditions. However, the intraday profile can be heterogeneous, as it is shown in Figure 2.3.

For purchases and sales, the opposite pattern is observed. The 24-hour output structure is selected for both targets, with validation rMAE values of 0.639 and 0.674, respectively. The corresponding 7-zone output structures are less accurate, with rMAE values of 0.719 for purchases and 0.806 for sales. The descriptive evidence in Figure 2.2 supports our previous interpretation, since it shows the strong scale heterogeneity of volume series across zones. In particular, in the 2024 validation period, the ratio between the largest and smallest average zonal level is 28.3 for purchases and 10.1 for sales, compared with only 1.06 for prices. This reflects the reality of the difference in the scales of these targets, which is consistent with the results.

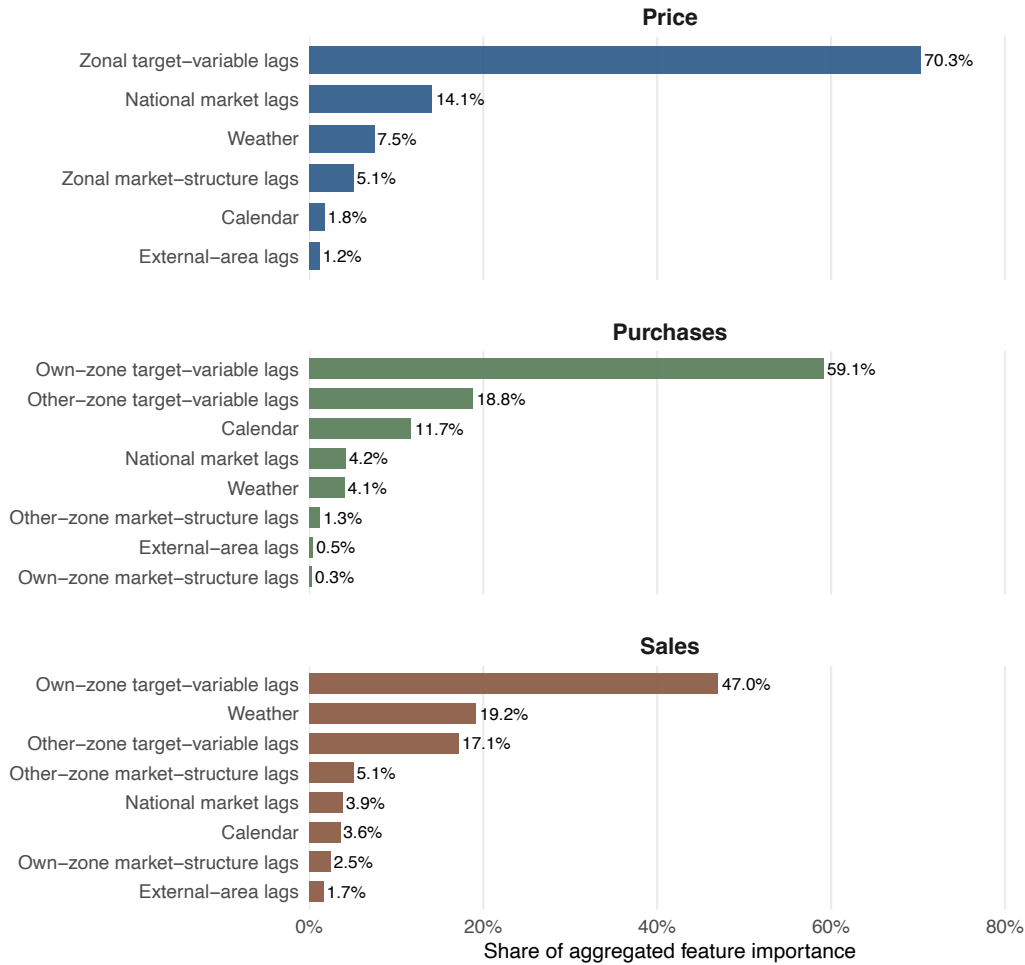
The final 2025 test results are reported in Table 4.7. Random Forest improves on the weekly naive benchmark for all targets. The average rMAE is 0.670 for prices, 0.622 for purchases and 0.670 for sales, corresponding to relative MAE reductions of approximately 33%, 38% and 33%, respectively. The improvement is also robust across zones, since all target–zone combinations have rMAE below one.

| Target    | MAE   | RMSE  | NMAE  | NRMSE | rMAE  |
|-----------|-------|-------|-------|-------|-------|
| Price     | 10.5  | 16.4  | –     | –     | 0.670 |
| Purchases | 152.0 | 251.0 | 0.034 | 0.056 | 0.622 |
| Sales     | 400.0 | 523.0 | 0.131 | 0.170 | 0.670 |

**TABLE 4.7.** Random Forest performance on the 2025 test set, averaged across zones.

Beyond predictive accuracy, Random Forest also offers a useful diagnostic tool, as it allows to inspect the fitted trees in order to assess which groups of predictors contribute most to the splitting decisions of the model. The feature importance is computed from the average reduction in the tree splitting criterion attributable to each predictor across the forest. This is also called impurity-based feature importance. The results are reported in Figure 4.2, where the importance values are aggregated by target.

The feature-importance results are consistent with the selected RF structures. In all targets, the most relevant predictors are lagged target variables, confirming that the forest mainly exploits temporal persistence and seasonal market patterns.



**Fig. 4.2.** Aggregated impurity-based Random Forest feature importance by target. Shares are normalised within each target.

For prices, the dominant block is formed by zonal target-variable lags. Since the selected price model predicts the seven zones jointly, this represents the lagged information from the full zonal system. For purchases and sales, own-zone target-variable lags are the most important group, while other-zone lags still provide additional information. This is coherent, volume forecasts rely primarily on the local daily profile of each target-zone series, complemented by regional lagged information. Weather variables also appear among the relevant groups, especially for sales, suggesting that the exogenous weather inputs add predictive information.

Overall, Random Forest performs very well in this application. It improves on the weekly naive benchmark for all targets and zones, while remaining computationally affordable under the monthly rolling recalibration scheme. In addition, the model also provides useful diagnostic information through feature importance, showing that lagged zonal information, local temporal patterns and weather variables are relevant for the forecasts. These diagnostics make Random Forest a valuable benchmark both predictively and descriptively.

## 4.4. XGBoost

The XGBoost model is the second model of reference in the machine learning family, described in Section 3.5. Unlike Random Forest, no multi-output structure is used here. A separate single-output model is estimated for each target–zone pair in long hourly format. The 24 day-ahead forecasts are obtained by applying the same fitted model to the 24 hourly feature rows of the delivery day. Temporal dependence is introduced through lagged predictors, while calendar variables, weather variables, national market variables and lagged cross-zone information are included as additional inputs.

The validation stage selected the same specification for the three targets. As shown in Table 4.8, the expanding window was selected in all cases, together with the hyperparameter configuration `xgb_medium_lr003`, which refers to a medium-complexity XGBoost configuration in the validation grid, while `lr003` indicates a learning rate of 0.03. This configuration corresponds to a moderate-depth boosting model with a low learning rate, subsampling and regularization. The result suggests that XGBoost benefits from using the full available historical sample without requiring very deep trees within the grid considered.

| Target    | Window    | Hyperparameter setting        | Validation rMAE |
|-----------|-----------|-------------------------------|-----------------|
| Price     | Expanding | <code>xgb_medium_lr003</code> | 0.712           |
| Purchases | Expanding | <code>xgb_medium_lr003</code> | 0.580           |
| Sales     | Expanding | <code>xgb_medium_lr003</code> | 0.632           |

**TABLE 4.8.** XGBoost strategy selected in validation by target.

The final 2025 test results are reported in Table 4.9. XGBoost improves on the weekly naive benchmark for all targets. The average rMAE is 0.704 for prices, 0.568 for purchases and 0.639 for sales, corresponding to relative MAE reductions of approximately 30%, 43% and 36%, respectively. The strongest relative performance is therefore obtained for purchase volumes.

| Target    | MAE   | RMSE  | NMAE  | NRMSE | rMAE  |
|-----------|-------|-------|-------|-------|-------|
| Price     | 11.1  | 18.9  | –     | –     | 0.704 |
| Purchases | 148.0 | 244.0 | 0.031 | 0.051 | 0.568 |
| Sales     | 383.0 | 498.0 | 0.126 | 0.164 | 0.639 |

**TABLE 4.9.** XGBoost performance on the 2025 test set, averaged across zones.

The zone-level results are also stable. For prices, rMAE ranges from 0.665 in SUD to 0.734 in CSUD. For purchases, all zones improve clearly on the weekly naive benchmark, with the best result in CALA (rMAE = 0.479) and the weakest in SARD (rMAE = 0.668). For sales, all zones also remain below one, with rMAE ranging from 0.533 in SUD to



0.702 in CSUD. Therefore, the good average performance is not driven by a small subset of zones. The complete zone-level results are reported in Table A.4 in the Appendix.

Feature importance provides an additional diagnostic of the fitted boosting models. In XGBoost, the reported importance is based on gain, that is, the improvement in the splitting criterion produced by the use of each predictor.

The importance pattern shows that XGBoost relies mainly on lagged market information. For prices, the largest contribution comes from other-zone target-variable lags, followed by own-target lags and national market lags. This is consistent with the strong connection between zonal prices, the PUN and the broader Italian market system. For purchases and sales, national market lags are the dominant group, followed by own-target lags and other-zone target-variable lags. The top individual variables are mostly weekly lags, such as `target_lag_168`, `sales_italy_lag_168` and `purchases_italy_lag_168`. This confirms that importance of the weekly lagged information that it has been suggested along the thesis. The ranking of the XGBoost features by gain is reported in Figure A.1 in the Appendix.

Overall, XGBoost proves to be a highly effective model in this application. It improves on the weekly naive benchmark for every target and zone, with particularly strong results for purchase and sales volumes, while still remaining computationally manageable under the monthly recalibration scheme. Furthermore, it provides a useful feature-importance diagnostic, that suggests that the model benefits from own-series persistence, other-zone market signals and national market aggregates information.

## 4.5. LSTM

The LSTM model was implemented as described in Subsection 3.6.2. The main specification is a direct 24-hour multi-output LSTM where, for each target–zone pair, the recurrent encoder processes a historical hourly sequence and a dense output head produces the full 24-hour profile of the delivery day. The input sequence includes own-zone market variables, common national market variables, cross-zone variables and calendar information, while delivery-day calendar and weather covariates are included as future-known inputs.

The first LSTM specification uses an expanding training window. This is the natural starting point for a neural network model, since it provides the largest available training sample. The validation stage selected the architecture separately by target. As shown in Table 4.10, the selected specifications use 64 hidden units and a learning rate of  $5 \times 10^{-4}$ , while the look-back length and number of recurrent layers differ across targets.

| Target    | Look-back | Hidden units | Layers | Batch size | Validation rMAE |
|-----------|-----------|--------------|--------|------------|-----------------|
| Price     | 168 h     | 64           | 2      | 32         | 1.010           |
| Purchases | 168 h     | 64           | 1      | 32         | 0.713           |
| Sales     | 336 h     | 64           | 2      | 32         | 0.722           |

**TABLE 4.10.** Direct expanding LSTM architecture selected in validation by target.

The validation results showed that the direct expanding LSTM was not equally effective across targets. It provided reasonable results for purchases and sales, but the price forecasts were much weaker and heterogeneous across zones. This motivated two additional LSTM experiments. First, the same direct 24-hour forecasting strategy was re-estimated using a rolling 24-month training window, with the aim of reducing the influence of older observations. This choice is particularly relevant for prices, since the exploratory analysis showed that the 2022 energy crisis was especially visible in the price series (Figure 2.1). Second, a recursive one-hour-ahead LSTM was estimated with a rolling 24-month window. This recursive model is more restrictive, but it provides a useful robustness check on the forecasting strategy.

The final 2025 test results are reported in Table 4.11. The direct expanding LSTM performs best for the two volume targets, with rMAE equal to 0.681 for purchases and 0.703 for sales. However, it does not improve on the weekly naive benchmark for prices, where the average rMAE is 1.011. The rolling-window extensions improve price forecasting, reducing the price rMAE to 0.913, and the recursive rolling 24-month model further reduces it to 0.887. However, these gains for prices do not extend to purchases and sales, where the direct expanding model remains the best LSTM specification.

| Model                 | Target    | MAE   | RMSE  | NMAE  | NRMSE | rMAE  |
|-----------------------|-----------|-------|-------|-------|-------|-------|
| Direct expanding      | Price     | 15.9  | 22.0  | –     | –     | 1.011 |
| Direct rolling 24m    | Price     | 14.3  | 20.3  | –     | –     | 0.913 |
| Recursive rolling 24m | Price     | 13.9  | 20.0  | –     | –     | 0.887 |
| Direct expanding      | Purchases | 163.0 | 251.0 | 0.037 | 0.057 | 0.681 |
| Direct rolling 24m    | Purchases | 190.0 | 290.0 | 0.041 | 0.063 | 0.751 |
| Recursive rolling 24m | Purchases | 196.0 | 284.0 | 0.039 | 0.061 | 0.723 |
| Direct expanding      | Sales     | 421.0 | 538.0 | 0.138 | 0.175 | 0.703 |
| Direct rolling 24m    | Sales     | 472.0 | 608.0 | 0.148 | 0.189 | 0.763 |
| Recursive rolling 24m | Sales     | 466.0 | 604.0 | 0.148 | 0.193 | 0.756 |

**TABLE 4.11.** LSTM performance on the 2025 test set, averaged across zones.

The zone-level results (that can be consulted in the Table A.4 in the Appendix) reinforce this interpretation. For the direct expanding LSTM, price performance varies substantially across zones, with rMAE above one in NORD, SARD and SUD. By contrast, the rolling

24-month direct specification keeps price rMAE below one in all zones, and the recursive rolling specification improves the average price result further, although it still performs poorly in CNOR. For purchases and sales, the direct expanding model is more stable, maintaining all zones below the weekly naive benchmark, and the relative gains are especially clear in the larger volume zones.

Altogether, LSTM provides useful evidence on the role of the forecasting strategy. The direct 24-hour expanding specification is the strongest LSTM approach for purchase and sales volumes, whereas price forecasting benefits from reducing the training window and, in this experiment, from the more restrictive recursive formulation. Nevertheless, the LSTM family is less consistently competitive than the tree-based models considered above.

## 4.6. Temporal Convolutional Network

The TCN model was implemented as described in Subsection 3.7.2. The causal dilated convolutional encoder processes a historical hourly sequence and a dense output head produces the full 24-hour delivery profile for each target–zone pair in the main specification. As in the LSTM implementation, the empirical analysis considers three specifications: a direct expanding TCN, a direct rolling 24-month TCN and a recursive rolling 24-month TCN.

The final 2025 test results are reported in Table 4.12. The table compares the direct expanding, direct rolling 24-month and recursive rolling 24-month TCN specifications on the same 2025 test set. Overall, all three specifications improve on the weekly naive benchmark on average, although the relative performance differs clearly across targets and training-window designs.

| Model                 | Target    | MAE   | RMSE  | NMAE  | NRMSE | rMAE  |
|-----------------------|-----------|-------|-------|-------|-------|-------|
| Direct expanding      | Price     | 14.6  | 20.7  | –     | –     | 0.932 |
| Direct rolling 24m    | Price     | 13.3  | 20.2  | –     | –     | 0.850 |
| Recursive rolling 24m | Price     | 13.2  | 19.4  | –     | –     | 0.842 |
| Direct expanding      | Purchases | 152.0 | 241.0 | 0.036 | 0.055 | 0.656 |
| Direct rolling 24m    | Purchases | 180.0 | 276.0 | 0.041 | 0.063 | 0.744 |
| Recursive rolling 24m | Purchases | 155.0 | 257.0 | 0.035 | 0.057 | 0.645 |
| Direct expanding      | Sales     | 391.0 | 502.0 | 0.128 | 0.164 | 0.652 |
| Direct rolling 24m    | Sales     | 486.0 | 707.0 | 0.152 | 0.210 | 0.778 |
| Recursive rolling 24m | Sales     | 435.0 | 571.0 | 0.140 | 0.182 | 0.713 |

**TABLE 4.12.** TCN performance on the 2025 test set, averaged across zones.

The effect of the rolling window is target-dependent. For prices, both rolling specifi-

cations improve substantially over the direct expanding TCN. The direct rolling 24-month model keeps rMAE below one in all zones, whereas the direct expanding TCN is worse than the weekly naive benchmark in CSUD. This suggests that the older observations included in the expanding sample are less helpful for price forecasting. For purchases and sales, the expanding model is more stable, although some zones remain more difficult. In particular, SARD is the weakest zone for purchase forecasts, while CSUD and NORD are among the more difficult zones for sales. This confirms that average target-level results hide relevant zonal heterogeneity. Detailed zone-level results are reported in Table A.4 in the appendix. Taken together, these results suggest that prices are more sensitive to the choice of training window, while volume targets benefit more from the larger historical sample, possibly as it captures more robust seasonal patterns.

In summary, the TCN results are stronger and more stable than the LSTM results. The direct expanding TCN performs well for volume forecasting, while price forecasts benefit from a more recent training sample. Therefore, the main conclusion is that TCNs provide useful nonlinear sequential forecasts, but their performance still depends strongly on the target and on the training-window design.

## 4.7. Final model comparison

After analysing each forecasting model separately, this section presents the final comparison across all methods. All models are compared on the same 2025 test set, using the weekly seasonal naive model as the reference benchmark for rMAE, as established in Section 3.1.2.

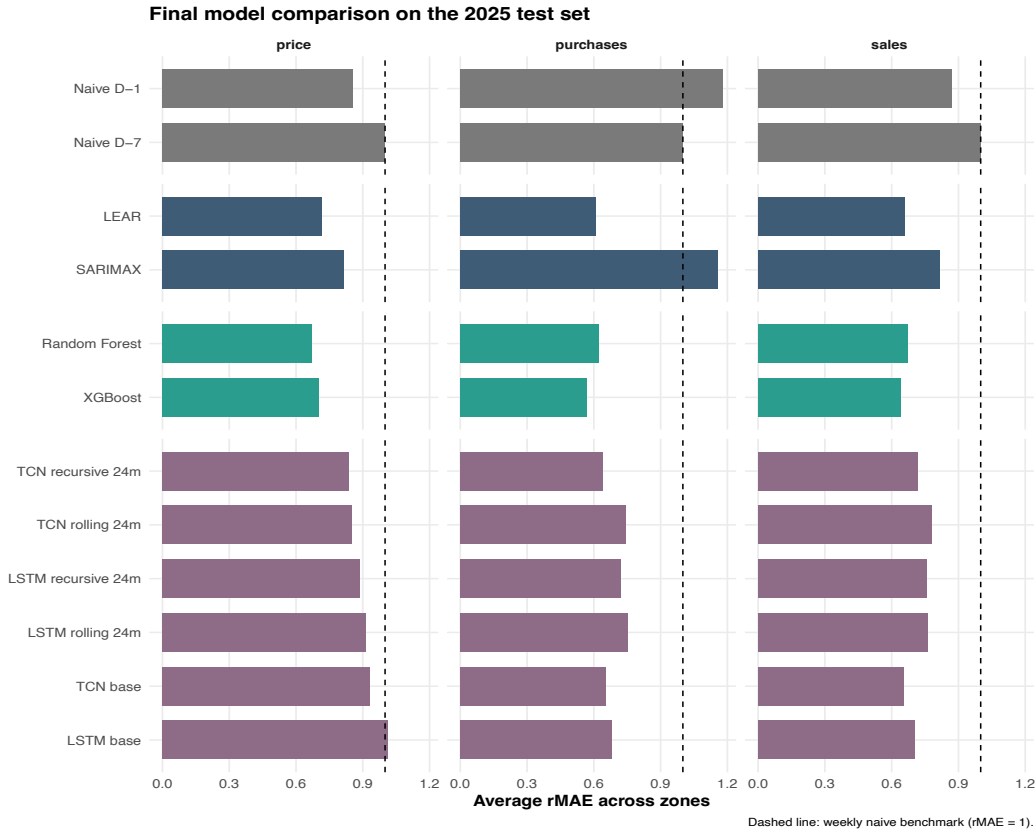
Table 4.13 summarizes the best-performing model for each target. The complete numerical comparison by model and target is reported in the Table A.2 in the Appendix. Overall, the best results are obtained by tree-based machine learning methods. Random Forest obtains the lowest average rMAE for prices, while XGBoost is the best model for purchases and sales. In all three cases, the best model improves substantially over the weekly naive benchmark.

| Target    | Best model    | MAE     | rMAE  | MAE improvement |
|-----------|---------------|---------|-------|-----------------|
| Price     | Random Forest | 10.537  | 0.670 | 33.0%           |
| Purchases | XGBoost       | 148.373 | 0.568 | 43.2%           |
| Sales     | XGBoost       | 382.987 | 0.639 | 36.1%           |

**TABLE 4.13.** Best model by target on the 2025 test set, averaged across zones.

Figure 4.3 provides the global ranking in terms of rMAE. For prices, Random Forest, XGBoost and LEAR form the strongest group of models, while the deep learning specifications remain competitive but do not dominate the ranking. This suggests that

the engineered lagged, calendar and market variables already capture most of the useful predictive structure. In this setting, the additional flexibility of LSTM and TCN models does not translate into better out-of-sample accuracy than simpler models based on these predictors.



**Fig. 4.3.** Final model comparison on the 2025 test set. The dashed line represents the weekly seasonal naive benchmark.

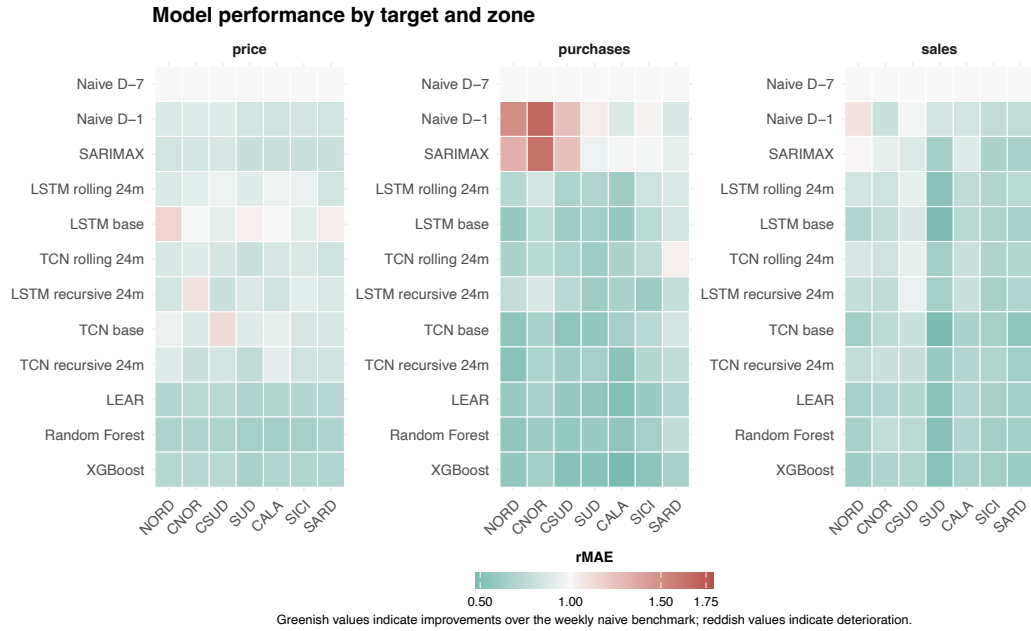
For purchases, XGBoost obtains the lowest rMAE, followed by LEAR, Random Forest and the recursive TCN. This confirms that purchase volumes are strongly driven by lagged market information, especially weekly patterns. In this target, models based on explicit lagged predictors perform adequately. On the other hand, the restricted specification of SARIMAX is not able to exploit this structure with the same effectiveness.

For sales, the ranking is more balanced. XGBoost remains the best model on average, but TCN, LEAR and Random Forest obtain similar results. This suggests that sales can be forecasted reasonably well with both tabular models and sequential neural networks. However, the fact that XGBoost remains the best model on average suggests that most of the useful predictive information is already contained in the engineered lagged, calendar and market variables.

The deep learning models generally improve over the weekly naive benchmark, but they are not the leading family in the final comparison. TCN models tend to perform better than LSTM models, especially for prices and sales. Nevertheless, their results are

more sensitive to the chosen training-window strategy. Overall, this suggests that the neural-network specifications are useful, but less stable than the best tree-based models in this empirical setting.

Figure 4.4 shows the model comparison by target and zone. The heatmap confirms that the main conclusions are not driven by a single bidding zone. Random Forest is the best price model in all zones. For purchases and sales, the ranking is more heterogeneous, although XGBoost remains the most robust model overall. This zonal heterogeneity is expected, since volumes depend more directly on regional demand.



**Fig. 4.4.** Model performance by target and zone on the 2025 test set. Values below one indicate improvement over the weekly seasonal naive benchmark.

Finally, the Diebold–Mariano tests presented in Section 3.1.2 are used here to complement the descriptive error metrics with statistical evidence. The tests are reported in the Table A.3 in the Appendix. In this comparison, they are mainly used to assess whether each model significantly improves over the weekly seasonal naive benchmark. The results support our previous analysis. The best tree-based models show statistically significant improvements over the benchmark across the three targets and most zones. By contrast, weaker specifications, such as SARIMAX for purchases and the base LSTM for prices, do not provide evidence of improvement.

Overall, the final comparison shows that the most accurate models in this study are not the most complex sequential neural networks, but the tree-based methods built on a rich set of lagged, zonal, national and calendar predictors. In the Italian day-ahead market dataset considered here, most of the forecastable structure appears to come from daily and weekly patterns, zonal interactions and aggregate market information. Random Forest and XGBoost are able to capture these signals effectively and with a very affordable computational cost.

## 5. Conclusions

---

This thesis has developed a comparative forecasting framework for the Italian day-ahead electricity market. The analysis focused on the seven physical GME bidding zones and considered three related forecasting targets: zonal prices, purchase volumes and sale volumes. The objective was not only to obtain accurate day-ahead forecasts, but also to compare different modelling families under a common empirical setting. For this reason, statistical models, tree-based machine learning methods and deep learning architectures were evaluated on the same market, targets, zones and hourly delivery horizons.

All models were compared using the same evaluation index, the same benchmark forecasts and the same out-of-sample test period. The use of a full validation year and a full test year makes the evaluation more robust, since model selection and final assessment are both based on periods that include different seasonal, weekly and market conditions. In particular, the weekly seasonal naive benchmark was used as the main reference for relative accuracy, since it captures one of the strongest empirical regularities of the data.

The empirical results show that the best overall performance is achieved by tree-based machine learning models. Random Forest provides the strongest results for price forecasting, while XGBoost performs best for purchase and sale volumes. These models improve clearly over the weekly naive benchmark, indicating that the Italian zonal market contains exploitable predictive structure beyond simple seasonal persistence. LEAR also behaves as a strong and stable forecast model, whereas SARIMAX provides a useful classical reference but is less competitive in the final comparison. The deep learning models improve on the naive benchmark in several cases, but they do not dominate the best machine learning specifications.

These results suggest that forecast accuracy in the Italian day-ahead market depends on the interaction between the target variable, the bidding zone and the modelling approach. There is not a single model that is uniformly dominant in every setting. Instead, the best results are obtained by models that are able to exploit the dominant temporal, zonal and market regularities of each forecasting problem.

The study is subject to some limitations. First, the empirical analysis is restricted to the hourly regime. This provides a consistent historical sample for the period 2021–2025, but the Italian day-ahead market moved to a quarter-hourly structure in October 2025. Extending the framework from 24 to 96 delivery periods per day is therefore a natural continuation of this work. Another relevant limitation is computational feasibility. A full hyperparameter search across all targets, zones, rolling origins and model families would be computationally expensive, especially for SARIMAX and deep learning models. For this reason, the thesis relies on validation-based model selection and compact hyperpa-

parameter grids. In addition, weather variables are included using historical delivery-day observations as forecast proxies, which is a reasonable approximation for a day-ahead setting but does not fully reproduce the uncertainty of real weather forecasts.

Finally, future research could be developed from this work. The most direct one is the transition to quarter-hourly forecasting, which would adapt the framework to the new Italian market regime. Another natural extension is the use of forecast combinations, for example by averaging or validation-weighting the predictions of the best statistical, machine learning and deep learning models, a strategy discussed in the EPF literature as forecast averaging or model averaging [1]. This could improve robustness if different models capture complementary aspects of the market. Future work could also consider richer multivariate or spatio-temporal specifications, especially for modelling cross-zone and cross-target interactions more explicitly.



# Bibliography

---

- [1] R. Weron, “Electricity price forecasting: A review of the state-of-the-art with a look into the future,” *International Journal of Forecasting*, vol. 30, no. 4, pp. 1030–1081, 2014. doi: [10.1016/j.ijforecast.2014.08.008](https://doi.org/10.1016/j.ijforecast.2014.08.008).
- [2] J. Lago, G. Marcjasz, B. De Schutter, and R. Weron, “Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark,” *Applied Energy*, vol. 293, p. 116983, 2021. doi: [10.1016/j.apenergy.2021.116983](https://doi.org/10.1016/j.apenergy.2021.116983). [Online]. Available: <http://dx.doi.org/10.1016/j.apenergy.2021.116983>.
- [3] Gestore dei Mercati Energetici, *Spot market (MPE) – Day-Ahead Market (MGP)*, <https://www.mercatoelettrico.org/en-us/Home/Markets/ElectricityMarket/Spot-Market-MPE>, Accessed: 2026-06-29, 2026.
- [4] Gestore dei Mercati Energetici, *Glossary*, <https://www.mercatoelettrico.org/en-us/Home/Glossary>, 2026.
- [5] D. Alessi, C. Bonaldo, and F. Fontini, “The impact of utility-scale RES power production on the Italian electricity prices,” *Italian Economic Journal*, vol. 12, pp. 477–511, 2026. doi: [10.1007/s40797-024-00309-y](https://doi.org/10.1007/s40797-024-00309-y). [Online]. Available: <https://doi.org/10.1007/s40797-024-00309-y>.
- [6] Gestore dei Mercati Energetici, *MGP historical data*, <https://www.mercatoelettrico.org/it-it/Home/Esiti/Elettricità/MGP/Statistiche/DatiStorici>, Accessed: 2026-07-01, 2026.
- [7] Gestore dei Mercati Energetici, *Vademecum of the Italian Power Exchange*, [https://www.mercatoelettrico.org/Portals/0/Documents/en-us/20250101VademecumBorsaElettrica\\_En.pdf](https://www.mercatoelettrico.org/Portals/0/Documents/en-us/20250101VademecumBorsaElettrica_En.pdf), Accessed: 2026-06-29, 2025.
- [8] Open-Meteo, *Historical weather api*, <https://open-meteo.com/en/docs/historical-weather-api>, Accessed: 2026-07-01, 2026.
- [9] C. Huurman, F. Ravazzolo, and C. Zhou, “The power of weather,” *Computational Statistics & Data Analysis*, vol. 56, no. 11, pp. 3793–3807, 2009. doi: [10.1016/j.csda.2010.06.021](https://doi.org/10.1016/j.csda.2010.06.021).
- [10] G. Marcjasz, T. Serafin, and R. Weron, “Selection of calibration windows for day-ahead electricity price forecasting,” *Energies*, vol. 11, no. 9, p. 2364, 2018. doi: [10.3390/en11092364](https://doi.org/10.3390/en11092364). [Online]. Available: <https://www.mdpi.com/1996-1073/11/9/2364>.

- [11] Council of the European Union, *Energy crisis: Three EU-coordinated measures to cut down bills*, <https://www.consilium.europa.eu/es/infographics/eu-measures-to-cut-down-energy-bills/>, Archived content. Last updated: 12 March 2025. Accessed: 2026-07-01, 2025.
- [12] M. Xie, C. Sandels, K. Zhu, and L. Nordström, “A seasonal ARIMA model with exogenous variables for elspot electricity prices in Sweden,” May 2013, pp. 1–4. doi: [10.1109/EEM.2013.6607293](https://doi.org/10.1109/EEM.2013.6607293).
- [13] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, Oct. 2001. doi: [10.1023/A:1010950718922](https://doi.org/10.1023/A:1010950718922).
- [14] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’16, ACM, Aug. 2016, pp. 785–794. doi: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- [15] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. doi: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- [16] S. Bai, J. Z. Kolter, and V. Koltun, *An empirical evaluation of generic convolutional and recurrent networks for sequence modeling*, 2018. arXiv: [1803.01271](https://arxiv.org/abs/1803.01271) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/1803.01271>.
- [17] M. U. Yousuf, I. Al-Bahadly, and E. Avci, “Current perspective on the accuracy of deterministic wind speed and power forecasting,” *IEEE Access*, vol. 7, pp. 159 547–159 564, 2019. doi: [10.1109/ACCESS.2019.2951153](https://doi.org/10.1109/ACCESS.2019.2951153).
- [18] A. Nespoli, E. Ogliari, S. Pretto, M. Gavazzeni, S. Vigani, and F. Paccanelli, “Electrical load forecast by means of LSTM: The impact of data quality,” *Forecasting*, vol. 3, no. 1, pp. 91–101, 2021. doi: [10.3390/forecast3010006](https://doi.org/10.3390/forecast3010006).
- [19] F. X. Diebold and R. S. Mariano, “Comparing predictive accuracy,” *Journal of Business & Economic Statistics*, vol. 13, no. 3, pp. 253–263, 1995. doi: [10.1080/07350015.1995.10524599](https://doi.org/10.1080/07350015.1995.10524599).
- [20] R. H. Shumway and D. S. Stoffer, *Time Series Analysis and Its Applications: With R Examples*, 4th. Springer, 2017.
- [21] R. J. Hyndman and Y. Khandakar, “Automatic time series forecasting: The forecast package for R,” *Journal of Statistical Software*, vol. 27, no. 3, pp. 1–22, 2008. doi: [10.18637/jss.v027.i03](https://doi.org/10.18637/jss.v027.i03).
- [22] R. J. Hyndman et al., *Forecast: Forecasting functions for time series and linear models*, R package version 9.0.2, 2026. doi: [10.32614/CRAN.package.forecast](https://doi.org/10.32614/CRAN.package.forecast). [Online]. Available: <https://CRAN.R-project.org/package=forecast>.

- [23] R. Tibshirani, “Regression shrinkage and selection via the lasso,” *Journal of the Royal Statistical Society: Series B*, vol. 58, no. 1, pp. 267–288, 1996. DOI: [10.1111/j.2517-6161.1996.tb02080.x](https://doi.org/10.1111/j.2517-6161.1996.tb02080.x).
- [24] F. Ziel, “Forecasting electricity spot prices using lasso: On capturing the autoregressive intraday structure,” *IEEE Transactions on Power Systems*, vol. 31, no. 6, pp. 4977–4987, Nov. 2016. DOI: [10.1109/tpwrs.2016.2521545](https://doi.org/10.1109/tpwrs.2016.2521545). [Online]. Available: <http://dx.doi.org/10.1109/TPWRS.2016.2521545>.
- [25] J. Friedman, T. Hastie, and R. Tibshirani, “Regularization paths for generalized linear models via coordinate descent,” *Journal of Statistical Software*, vol. 33, no. 1, pp. 1–22, 2010. DOI: [10.18637/jss.v033.i01](https://doi.org/10.18637/jss.v033.i01).
- [26] M. R. Segal and Y. Xiao, “Multivariate random forests,” *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 1, no. 1, pp. 80–87, 2011. DOI: [10.1002/widm.12](https://doi.org/10.1002/widm.12).
- [27] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [28] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [29] A. Paszke et al., “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [30] S. Ben Taieb, G. Bontempi, A. Sorjamaa, and A. Lendasse, “Long-term prediction of time series by combining direct and mimo strategies,” in *2009 International Joint Conference on Neural Networks*, IEEE, 2009, pp. 3054–3061. DOI: [10.1109/IJCNN.2009.5178962](https://doi.org/10.1109/IJCNN.2009.5178962).

# A. Additional results

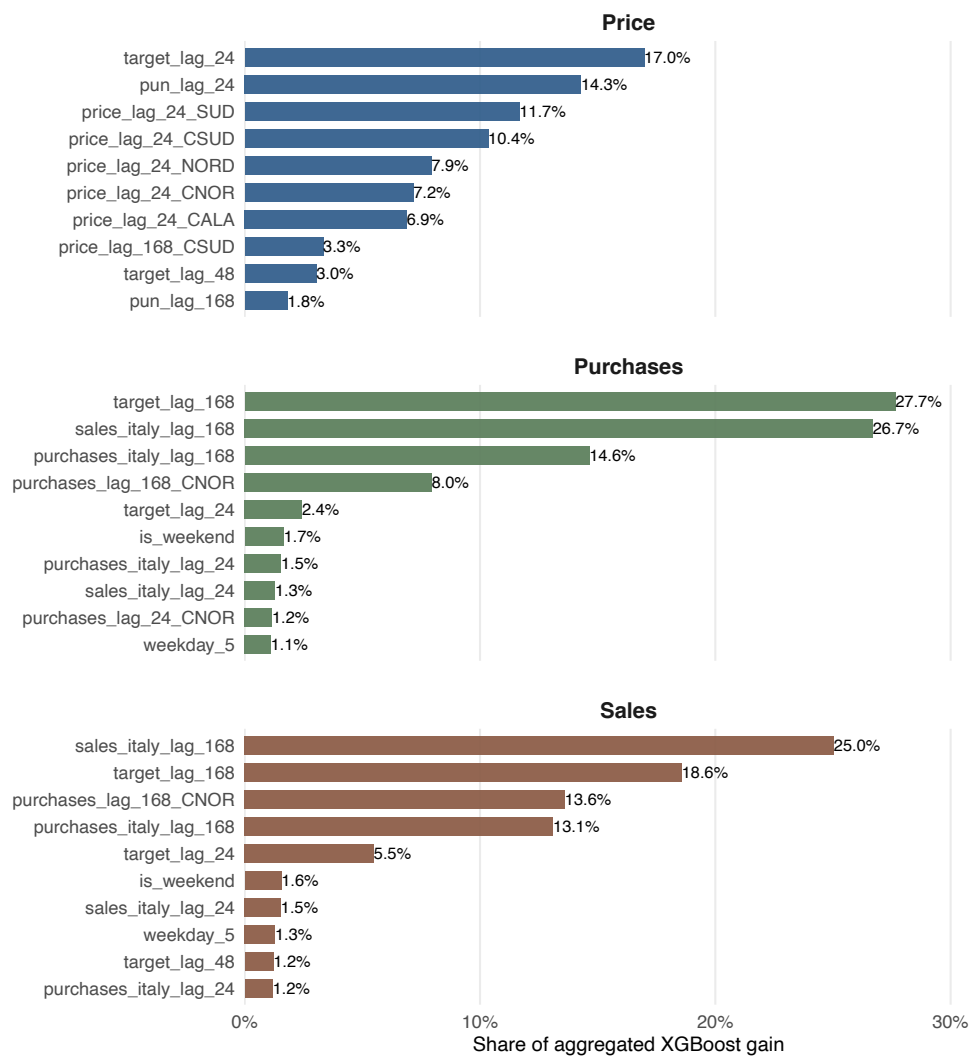
---

## A.1. Weekly persistence diagnostics

| Zone | Correlation at lag 168h |           |       |
|------|-------------------------|-----------|-------|
|      | Price                   | Purchases | Sales |
| NORD | 0.759                   | 0.882     | 0.831 |
| CNOR | 0.763                   | 0.927     | 0.781 |
| CSUD | 0.757                   | 0.930     | 0.815 |
| SUD  | 0.694                   | 0.898     | 0.546 |
| CALA | 0.681                   | 0.945     | 0.685 |
| SICI | 0.674                   | 0.933     | 0.651 |
| SARD | 0.642                   | 0.779     | 0.682 |

**TABLE A.1.** Weekly persistence diagnostics by zone in the 2025 test period.  
Correlations are computed with the same variable lagged by 168 hours.

## A.2. XGBoost feature-importance diagnostics



**Fig. A.1.** Top 10 individual XGBoost features by target, ranked by aggregated gain.

### A.3. Final model comparison results

**TABLE A.2.** Final test performance by model and target, averaged across zones.

| Model              | Family           | Target    | MAE     | RMSE    | NMAE  | NRMSE | rMAE  | impMAE(%) |
|--------------------|------------------|-----------|---------|---------|-------|-------|-------|-----------|
| Random Forest      | Machine learning | price     | 10.537  | 16.384  | –     | –     | 0.670 | 33.030    |
| XGBoost            | Machine learning | price     | 11.062  | 18.872  | –     | –     | 0.704 | 29.618    |
| LEAR               | Statistical      | price     | 11.267  | 17.411  | –     | –     | 0.716 | 28.397    |
| SARIMAX            | Statistical      | price     | 12.827  | 20.311  | –     | –     | 0.817 | 18.331    |
| TCN recursive 24m  | Deep learning    | price     | 13.162  | 19.364  | –     | –     | 0.837 | 16.345    |
| TCN rolling 24m    | Deep learning    | price     | 13.350  | 20.235  | –     | –     | 0.850 | 15.036    |
| Naive D-1          | Benchmark        | price     | 13.439  | 22.190  | –     | –     | 0.856 | 14.389    |
| LSTM recursive 24m | Deep learning    | price     | 13.948  | 19.977  | –     | –     | 0.887 | 11.289    |
| LSTM rolling 24m   | Deep learning    | price     | 14.343  | 20.334  | –     | –     | 0.913 | 8.722     |
| TCN base           | Deep learning    | price     | 14.586  | 20.692  | –     | –     | 0.932 | 6.819     |
| Naive D-7          | Benchmark        | price     | 15.744  | 24.489  | –     | –     | 1.000 | 0.000     |
| LSTM base          | Deep learning    | price     | 15.901  | 22.038  | –     | –     | 1.011 | -1.132    |
| XGBoost            | Machine learning | purchases | 148.373 | 243.574 | 0.031 | 0.051 | 0.568 | 43.253    |
| LEAR               | Statistical      | purchases | 157.345 | 262.207 | 0.033 | 0.055 | 0.608 | 39.246    |
| Random Forest      | Machine learning | purchases | 152.407 | 250.960 | 0.034 | 0.056 | 0.622 | 37.785    |
| TCN recursive 24m  | Deep learning    | purchases | 151.251 | 249.584 | 0.035 | 0.056 | 0.642 | 35.825    |
| TCN base           | Deep learning    | purchases | 152.291 | 241.469 | 0.036 | 0.055 | 0.656 | 34.432    |
| LSTM base          | Deep learning    | purchases | 162.502 | 250.845 | 0.037 | 0.057 | 0.681 | 31.923    |
| LSTM recursive 24m | Deep learning    | purchases | 195.919 | 284.359 | 0.039 | 0.061 | 0.723 | 27.683    |
| TCN rolling 24m    | Deep learning    | purchases | 180.098 | 276.095 | 0.041 | 0.063 | 0.744 | 25.623    |
| LSTM rolling 24m   | Deep learning    | purchases | 190.064 | 290.231 | 0.041 | 0.062 | 0.751 | 24.873    |
| Naive D-7          | Benchmark        | purchases | 258.709 | 421.931 | 0.054 | 0.086 | 1.000 | 0.000     |
| SARIMAX            | Statistical      | purchases | 332.842 | 527.846 | 0.062 | 0.095 | 1.156 | -15.604   |
| Naive D-1          | Benchmark        | purchases | 361.947 | 568.087 | 0.064 | 0.100 | 1.180 | -18.029   |
| XGBoost            | Machine learning | sales     | 382.987 | 498.388 | 0.126 | 0.164 | 0.639 | 36.071    |
| TCN base           | Deep learning    | sales     | 390.627 | 501.688 | 0.128 | 0.164 | 0.652 | 34.748    |
| LEAR               | Statistical      | sales     | 395.305 | 512.127 | 0.130 | 0.168 | 0.658 | 34.176    |
| Random Forest      | Machine learning | sales     | 400.170 | 522.801 | 0.131 | 0.170 | 0.670 | 33.017    |
| LSTM base          | Deep learning    | sales     | 420.768 | 537.692 | 0.138 | 0.175 | 0.703 | 29.743    |
| TCN recursive 24m  | Deep learning    | sales     | 442.720 | 581.259 | 0.140 | 0.182 | 0.715 | 28.503    |
| LSTM recursive 24m | Deep learning    | sales     | 465.618 | 604.405 | 0.148 | 0.193 | 0.756 | 24.414    |
| LSTM rolling 24m   | Deep learning    | sales     | 471.568 | 607.914 | 0.148 | 0.189 | 0.763 | 23.677    |
| TCN rolling 24m    | Deep learning    | sales     | 486.291 | 707.232 | 0.152 | 0.210 | 0.778 | 22.196    |
| SARIMAX            | Statistical      | sales     | 526.052 | 716.188 | 0.158 | 0.208 | 0.816 | 18.443    |
| Naive D-1          | Benchmark        | sales     | 576.265 | 803.343 | 0.169 | 0.232 | 0.870 | 13.012    |
| Naive D-7          | Benchmark        | sales     | 610.126 | 796.854 | 0.198 | 0.260 | 1.000 | 0.000     |

**TABLE A.3.** Summary of zone-level Diebold–Mariano tests against the weekly seasonal naive benchmark. Each row summarizes the seven tests obtained for the seven physical bidding zones.

| Model              | Family           | Target    | Sign. better | Not sign. | Sign. worse | Mean rMAE |
|--------------------|------------------|-----------|--------------|-----------|-------------|-----------|
| Random Forest      | Machine learning | price     | 7            | 0         | 0           | 0.670     |
| XGBoost            | Machine learning | price     | 7            | 0         | 0           | 0.704     |
| LEAR               | Statistical      | price     | 7            | 0         | 0           | 0.716     |
| SARIMAX            | Statistical      | price     | 7            | 0         | 0           | 0.817     |
| TCN recursive 24m  | Deep learning    | price     | 7            | 0         | 0           | 0.837     |
| TCN rolling 24m    | Deep learning    | price     | 7            | 0         | 0           | 0.850     |
| LSTM rolling 24m   | Deep learning    | price     | 7            | 0         | 0           | 0.913     |
| LSTM recursive 24m | Deep learning    | price     | 6            | 0         | 1           | 0.887     |
| TCN base           | Deep learning    | price     | 6            | 0         | 1           | 0.932     |
| LSTM base          | Deep learning    | price     | 2            | 2         | 3           | 1.011     |
| XGBoost            | Machine learning | purchases | 7            | 0         | 0           | 0.568     |
| LEAR               | Statistical      | purchases | 7            | 0         | 0           | 0.608     |
| Random Forest      | Machine learning | purchases | 7            | 0         | 0           | 0.622     |
| TCN recursive 24m  | Deep learning    | purchases | 7            | 0         | 0           | 0.642     |
| TCN base           | Deep learning    | purchases | 7            | 0         | 0           | 0.656     |
| LSTM base          | Deep learning    | purchases | 7            | 0         | 0           | 0.681     |
| LSTM recursive 24m | Deep learning    | purchases | 7            | 0         | 0           | 0.723     |
| LSTM rolling 24m   | Deep learning    | purchases | 7            | 0         | 0           | 0.751     |
| TCN rolling 24m    | Deep learning    | purchases | 6            | 0         | 1           | 0.744     |
| SARIMAX            | Statistical      | purchases | 2            | 2         | 3           | 1.156     |
| XGBoost            | Machine learning | sales     | 7            | 0         | 0           | 0.639     |
| TCN base           | Deep learning    | sales     | 7            | 0         | 0           | 0.652     |
| LEAR               | Statistical      | sales     | 7            | 0         | 0           | 0.658     |
| Random Forest      | Machine learning | sales     | 7            | 0         | 0           | 0.670     |
| LSTM base          | Deep learning    | sales     | 7            | 0         | 0           | 0.703     |
| TCN recursive 24m  | Deep learning    | sales     | 7            | 0         | 0           | 0.715     |
| LSTM recursive 24m | Deep learning    | sales     | 7            | 0         | 0           | 0.756     |
| LSTM rolling 24m   | Deep learning    | sales     | 7            | 0         | 0           | 0.763     |
| TCN rolling 24m    | Deep learning    | sales     | 7            | 0         | 0           | 0.778     |
| SARIMAX            | Statistical      | sales     | 6            | 1         | 0           | 0.816     |

**TABLE A.4.** Complete test performance by model, target and zone.

| Model              | Family           | Target | Zone | MAE    | RMSE   | NMAE | NRMSE | rMAE  | impMAE(%) |
|--------------------|------------------|--------|------|--------|--------|------|-------|-------|-----------|
| Random Forest      | Machine learning | price  | NORD | 9.485  | 14.444 | –    | –     | 0.682 | 31.782    |
| LEAR               | Statistical      | price  | NORD | 9.929  | 14.771 | –    | –     | 0.714 | 28.584    |
| XGBoost            | Machine learning | price  | NORD | 9.953  | 18.978 | –    | –     | 0.716 | 28.414    |
| SARIMAX            | Statistical      | price  | NORD | 11.499 | 18.040 | –    | –     | 0.827 | 17.294    |
| LSTM recursive 24m | Deep learning    | price  | NORD | 11.607 | 16.068 | –    | –     | 0.835 | 16.520    |
| TCN rolling 24m    | Deep learning    | price  | NORD | 12.009 | 19.370 | –    | –     | 0.864 | 13.626    |
| Naive D-1          | Benchmark        | price  | NORD | 12.206 | 19.696 | –    | –     | 0.878 | 12.214    |
| LSTM rolling 24m   | Deep learning    | price  | NORD | 12.209 | 17.006 | –    | –     | 0.878 | 12.192    |
| TCN recursive 24m  | Deep learning    | price  | NORD | 12.366 | 17.196 | –    | –     | 0.889 | 11.056    |
| TCN base           | Deep learning    | price  | NORD | 13.232 | 18.566 | –    | –     | 0.952 | 4.832     |
| Naive D-7          | Benchmark        | price  | NORD | 13.904 | 20.491 | –    | –     | 1.000 | 0.000     |
| LSTM base          | Deep learning    | price  | NORD | 16.212 | 20.729 | –    | –     | 1.166 | -16.599   |
| Random Forest      | Machine learning | price  | CNOR | 9.692  | 14.613 | –    | –     | 0.687 | 31.307    |
| XGBoost            | Machine learning | price  | CNOR | 10.168 | 19.719 | –    | –     | 0.721 | 27.937    |
| LEAR               | Statistical      | price  | CNOR | 10.321 | 15.206 | –    | –     | 0.732 | 26.851    |
| TCN recursive 24m  | Deep learning    | price  | CNOR | 11.275 | 16.162 | –    | –     | 0.799 | 20.087    |
| SARIMAX            | Statistical      | price  | CNOR | 12.014 | 18.624 | –    | –     | 0.852 | 14.854    |
| TCN base           | Deep learning    | price  | CNOR | 12.348 | 17.513 | –    | –     | 0.875 | 12.485    |
| Naive D-1          | Benchmark        | price  | CNOR | 12.536 | 19.967 | –    | –     | 0.888 | 11.153    |
| TCN rolling 24m    | Deep learning    | price  | CNOR | 12.677 | 18.516 | –    | –     | 0.898 | 10.156    |
| LSTM rolling 24m   | Deep learning    | price  | CNOR | 12.943 | 17.490 | –    | –     | 0.917 | 8.271     |
| LSTM base          | Deep learning    | price  | CNOR | 14.012 | 18.991 | –    | –     | 0.993 | 0.696     |
| Naive D-7          | Benchmark        | price  | CNOR | 14.110 | 20.728 | –    | –     | 1.000 | 0.000     |
| LSTM recursive 24m | Deep learning    | price  | CNOR | 15.419 | 21.639 | –    | –     | 1.093 | -9.277    |
| Random Forest      | Machine learning | price  | CSUD | 9.743  | 14.693 | –    | –     | 0.684 | 31.589    |
| XGBoost            | Machine learning | price  | CSUD | 10.451 | 17.378 | –    | –     | 0.734 | 26.622    |
| LEAR               | Statistical      | price  | CSUD | 10.504 | 15.724 | –    | –     | 0.738 | 26.247    |
| LSTM recursive 24m | Deep learning    | price  | CSUD | 11.378 | 16.034 | –    | –     | 0.799 | 20.114    |
| TCN recursive 24m  | Deep learning    | price  | CSUD | 11.892 | 17.483 | –    | –     | 0.835 | 16.505    |
| TCN rolling 24m    | Deep learning    | price  | CSUD | 12.097 | 17.444 | –    | –     | 0.849 | 15.065    |
| SARIMAX            | Statistical      | price  | CSUD | 12.138 | 18.860 | –    | –     | 0.852 | 14.774    |
| Naive D-1          | Benchmark        | price  | CSUD | 12.779 | 20.601 | –    | –     | 0.897 | 10.275    |
| LSTM base          | Deep learning    | price  | CSUD | 13.219 | 18.901 | –    | –     | 0.928 | 7.186     |
| LSTM rolling 24m   | Deep learning    | price  | CSUD | 13.559 | 18.568 | –    | –     | 0.952 | 4.800     |
| Naive D-7          | Benchmark        | price  | CSUD | 14.242 | 21.264 | –    | –     | 1.000 | 0.000     |
| TCN base           | Deep learning    | price  | CSUD | 16.181 | 21.590 | –    | –     | 1.136 | -13.610   |
| Random Forest      | Machine learning | price  | SUD  | 10.223 | 15.874 | –    | –     | 0.647 | 35.312    |
| XGBoost            | Machine learning | price  | SUD  | 10.516 | 16.562 | –    | –     | 0.665 | 33.458    |
| LEAR               | Statistical      | price  | SUD  | 10.783 | 16.476 | –    | –     | 0.682 | 31.769    |
| TCN recursive 24m  | Deep learning    | price  | SUD  | 11.956 | 17.691 | –    | –     | 0.756 | 24.348    |
| SARIMAX            | Statistical      | price  | SUD  | 12.502 | 19.794 | –    | –     | 0.791 | 20.890    |
| TCN rolling 24m    | Deep learning    | price  | SUD  | 12.617 | 18.804 | –    | –     | 0.798 | 20.161    |
| Naive D-1          | Benchmark        | price  | SUD  | 13.191 | 21.735 | –    | –     | 0.835 | 16.527    |
| LSTM recursive 24m | Deep learning    | price  | SUD  | 13.757 | 18.958 | –    | –     | 0.871 | 12.948    |
| LSTM rolling 24m   | Deep learning    | price  | SUD  | 14.324 | 20.280 | –    | –     | 0.906 | 9.359     |
| TCN base           | Deep learning    | price  | SUD  | 14.325 | 20.441 | –    | –     | 0.906 | 9.351     |
| Naive D-7          | Benchmark        | price  | SUD  | 15.803 | 25.134 | –    | –     | 1.000 | 0.000     |
| LSTM base          | Deep learning    | price  | SUD  | 16.322 | 22.485 | –    | –     | 1.033 | -3.282    |
| Random Forest      | Machine learning | price  | CALA | 10.321 | 15.928 | –    | –     | 0.643 | 35.653    |
| XGBoost            | Machine learning | price  | CALA | 11.182 | 18.018 | –    | –     | 0.697 | 30.286    |
| LEAR               | Statistical      | price  | CALA | 11.387 | 17.696 | –    | –     | 0.710 | 29.002    |
| SARIMAX            | Statistical      | price  | CALA | 12.777 | 20.143 | –    | –     | 0.797 | 20.338    |
| LSTM recursive 24m | Deep learning    | price  | CALA | 13.205 | 19.203 | –    | –     | 0.823 | 17.669    |
| Naive D-1          | Benchmark        | price  | CALA | 13.257 | 21.803 | –    | –     | 0.827 | 17.348    |
| TCN rolling 24m    | Deep learning    | price  | CALA | 13.679 | 20.303 | –    | –     | 0.853 | 14.716    |
| TCN recursive 24m  | Deep learning    | price  | CALA | 14.826 | 21.600 | –    | –     | 0.924 | 7.566     |
| TCN base           | Deep learning    | price  | CALA | 15.034 | 21.727 | –    | –     | 0.937 | 6.264     |
| LSTM rolling 24m   | Deep learning    | price  | CALA | 15.366 | 21.689 | –    | –     | 0.958 | 4.196     |
| LSTM base          | Deep learning    | price  | CALA | 15.937 | 22.672 | –    | –     | 0.994 | 0.637     |
| Naive D-7          | Benchmark        | price  | CALA | 16.039 | 25.369 | –    | –     | 1.000 | 0.000     |



| Model              | Family           | Target    | Zone | MAE      | RMSE     | NMAE  | NRMSE | rMAE  | impMAE(%) |
|--------------------|------------------|-----------|------|----------|----------|-------|-------|-------|-----------|
| Random Forest      | Machine learning | price     | SICI | 11.307   | 17.783   | –     | –     | 0.657 | 34.301    |
| XGBoost            | Machine learning | price     | SICI | 12.019   | 19.604   | –     | –     | 0.698 | 30.161    |
| LEAR               | Statistical      | price     | SICI | 12.278   | 19.292   | –     | –     | 0.713 | 28.656    |
| SARIMAX            | Statistical      | price     | SICI | 13.695   | 21.857   | –     | –     | 0.796 | 20.421    |
| TCN recursive 24m  | Deep learning    | price     | SICI | 14.198   | 21.188   | –     | –     | 0.825 | 17.498    |
| Naive D-1          | Benchmark        | price     | SICI | 14.411   | 24.081   | –     | –     | 0.837 | 16.267    |
| TCN base           | Deep learning    | price     | SICI | 14.695   | 21.746   | –     | –     | 0.854 | 14.613    |
| TCN rolling 24m    | Deep learning    | price     | SICI | 14.981   | 23.295   | –     | –     | 0.871 | 12.949    |
| LSTM base          | Deep learning    | price     | SICI | 15.705   | 23.029   | –     | –     | 0.912 | 8.746     |
| LSTM recursive 24m | Deep learning    | price     | SICI | 15.785   | 23.299   | –     | –     | 0.917 | 8.278     |
| LSTM rolling 24m   | Deep learning    | price     | SICI | 16.224   | 23.150   | –     | –     | 0.943 | 5.730     |
| Naive D-7          | Benchmark        | price     | SICI | 17.210   | 26.879   | –     | –     | 1.000 | 0.000     |
| Random Forest      | Machine learning | price     | SARD | 12.989   | 21.356   | –     | –     | 0.687 | 31.266    |
| XGBoost            | Machine learning | price     | SARD | 13.143   | 21.843   | –     | –     | 0.696 | 30.446    |
| LEAR               | Statistical      | price     | SARD | 13.669   | 22.711   | –     | –     | 0.723 | 27.668    |
| SARIMAX            | Statistical      | price     | SARD | 15.165   | 24.858   | –     | –     | 0.802 | 19.748    |
| TCN rolling 24m    | Deep learning    | price     | SARD | 15.386   | 23.912   | –     | –     | 0.814 | 18.577    |
| TCN recursive 24m  | Deep learning    | price     | SARD | 15.617   | 24.227   | –     | –     | 0.826 | 17.357    |
| Naive D-1          | Benchmark        | price     | SARD | 15.696   | 27.449   | –     | –     | 0.831 | 16.939    |
| LSTM rolling 24m   | Deep learning    | price     | SARD | 15.778   | 24.156   | –     | –     | 0.835 | 16.505    |
| TCN base           | Deep learning    | price     | SARD | 16.290   | 23.260   | –     | –     | 0.862 | 13.796    |
| LSTM recursive 24m | Deep learning    | price     | SARD | 16.483   | 24.642   | –     | –     | 0.872 | 12.774    |
| Naive D-7          | Benchmark        | price     | SARD | 18.897   | 31.560   | –     | –     | 1.000 | 0.000     |
| LSTM base          | Deep learning    | price     | SARD | 19.900   | 27.462   | –     | –     | 1.053 | -5.310    |
| TCN recursive 24m  | Deep learning    | purchases | NORD | 595.038  | 1031.792 | 0.033 | 0.058 | 0.538 | 46.201    |
| TCN base           | Deep learning    | purchases | NORD | 617.647  | 1026.571 | 0.034 | 0.057 | 0.558 | 44.157    |
| Random Forest      | Machine learning | purchases | NORD | 631.685  | 1059.992 | 0.035 | 0.059 | 0.571 | 42.888    |
| XGBoost            | Machine learning | purchases | NORD | 641.602  | 1063.404 | 0.036 | 0.059 | 0.580 | 41.992    |
| LSTM base          | Deep learning    | purchases | NORD | 656.632  | 1045.415 | 0.037 | 0.058 | 0.594 | 40.633    |
| LEAR               | Statistical      | purchases | NORD | 669.587  | 1136.416 | 0.037 | 0.063 | 0.605 | 39.461    |
| TCN rolling 24m    | Deep learning    | purchases | NORD | 747.598  | 1144.735 | 0.042 | 0.064 | 0.676 | 32.408    |
| LSTM rolling 24m   | Deep learning    | purchases | NORD | 803.521  | 1250.985 | 0.045 | 0.070 | 0.727 | 27.352    |
| LSTM recursive 24m | Deep learning    | purchases | NORD | 855.171  | 1216.645 | 0.048 | 0.068 | 0.773 | 22.682    |
| Naive D-7          | Benchmark        | purchases | NORD | 1106.048 | 1847.849 | 0.062 | 0.103 | 1.000 | 0.000     |
| SARIMAX            | Statistical      | purchases | NORD | 1474.748 | 2417.407 | 0.082 | 0.135 | 1.333 | -33.335   |
| Naive D-1          | Benchmark        | purchases | NORD | 1665.572 | 2622.392 | 0.093 | 0.146 | 1.506 | -50.588   |
| Random Forest      | Machine learning | purchases | CNOR | 87.162   | 145.952  | 0.032 | 0.053 | 0.618 | 38.183    |
| XGBoost            | Machine learning | purchases | CNOR | 89.019   | 146.678  | 0.032 | 0.053 | 0.631 | 36.866    |
| TCN base           | Deep learning    | purchases | CNOR | 92.925   | 140.260  | 0.034 | 0.051 | 0.659 | 34.095    |
| LEAR               | Statistical      | purchases | CNOR | 95.082   | 158.463  | 0.034 | 0.058 | 0.674 | 32.566    |
| TCN recursive 24m  | Deep learning    | purchases | CNOR | 95.917   | 154.696  | 0.035 | 0.056 | 0.680 | 31.973    |
| TCN rolling 24m    | Deep learning    | purchases | CNOR | 104.124  | 168.172  | 0.038 | 0.061 | 0.739 | 26.152    |
| LSTM base          | Deep learning    | purchases | CNOR | 104.564  | 154.113  | 0.038 | 0.056 | 0.742 | 25.840    |
| LSTM rolling 24m   | Deep learning    | purchases | CNOR | 118.525  | 190.984  | 0.043 | 0.069 | 0.841 | 15.939    |
| LSTM recursive 24m | Deep learning    | purchases | CNOR | 121.765  | 175.085  | 0.044 | 0.064 | 0.864 | 13.641    |
| Naive D-7          | Benchmark        | purchases | CNOR | 140.999  | 235.834  | 0.051 | 0.086 | 1.000 | 0.000     |
| SARIMAX            | Statistical      | purchases | CNOR | 230.245  | 365.020  | 0.084 | 0.132 | 1.633 | -63.296   |
| Naive D-1          | Benchmark        | purchases | CNOR | 237.440  | 388.943  | 0.086 | 0.141 | 1.684 | -68.398   |
| XGBoost            | Machine learning | purchases | CSUD | 145.342  | 233.225  | 0.025 | 0.041 | 0.527 | 47.309    |
| TCN base           | Deep learning    | purchases | CSUD | 155.283  | 225.006  | 0.027 | 0.039 | 0.563 | 43.705    |
| Random Forest      | Machine learning | purchases | CSUD | 161.100  | 255.140  | 0.028 | 0.045 | 0.584 | 41.596    |
| LEAR               | Statistical      | purchases | CSUD | 163.153  | 261.382  | 0.029 | 0.046 | 0.592 | 40.852    |
| LSTM base          | Deep learning    | purchases | CSUD | 172.210  | 248.167  | 0.030 | 0.044 | 0.624 | 37.568    |
| TCN recursive 24m  | Deep learning    | purchases | CSUD | 172.485  | 258.656  | 0.030 | 0.045 | 0.625 | 37.468    |
| TCN rolling 24m    | Deep learning    | purchases | CSUD | 187.761  | 289.379  | 0.033 | 0.051 | 0.681 | 31.930    |
| LSTM rolling 24m   | Deep learning    | purchases | CSUD | 187.943  | 265.009  | 0.033 | 0.046 | 0.681 | 31.864    |
| LSTM recursive 24m | Deep learning    | purchases | CSUD | 204.177  | 295.181  | 0.036 | 0.052 | 0.740 | 25.979    |
| Naive D-7          | Benchmark        | purchases | CSUD | 275.836  | 419.650  | 0.048 | 0.074 | 1.000 | 0.000     |
| SARIMAX            | Statistical      | purchases | CSUD | 347.179  | 508.507  | 0.061 | 0.089 | 1.259 | -25.864   |
| Naive D-1          | Benchmark        | purchases | CSUD | 348.053  | 532.747  | 0.061 | 0.093 | 1.262 | -26.181   |
| XGBoost            | Machine learning | purchases | SUD  | 56.663   | 83.733   | 0.028 | 0.042 | 0.530 | 46.965    |
| LEAR               | Statistical      | purchases | SUD  | 60.425   | 91.771   | 0.030 | 0.046 | 0.566 | 43.444    |

| Model              | Family           | Target    | Zone | MAE      | RMSE     | NMAE  | NRMSE | rMAE  | impMAE(%) |
|--------------------|------------------|-----------|------|----------|----------|-------|-------|-------|-----------|
| TCN base           | Deep learning    | purchases | SUD  | 61.903   | 85.683   | 0.031 | 0.043 | 0.579 | 42.060    |
| Random Forest      | Machine learning | purchases | SUD  | 64.357   | 93.670   | 0.032 | 0.047 | 0.602 | 39.764    |
| LSTM recursive 24m | Deep learning    | purchases | SUD  | 66.244   | 97.106   | 0.033 | 0.049 | 0.620 | 37.997    |
| TCN rolling 24m    | Deep learning    | purchases | SUD  | 66.528   | 91.662   | 0.033 | 0.046 | 0.623 | 37.732    |
| LSTM base          | Deep learning    | purchases | SUD  | 67.907   | 94.257   | 0.034 | 0.047 | 0.636 | 36.440    |
| TCN recursive 24m  | Deep learning    | purchases | SUD  | 68.850   | 96.799   | 0.034 | 0.048 | 0.644 | 35.558    |
| LSTM rolling 24m   | Deep learning    | purchases | SUD  | 74.727   | 101.141  | 0.037 | 0.051 | 0.699 | 30.057    |
| SARIMAX            | Statistical      | purchases | SUD  | 102.036  | 142.689  | 0.051 | 0.071 | 0.955 | 4.496     |
| Naive D-7          | Benchmark        | purchases | SUD  | 106.840  | 160.420  | 0.053 | 0.080 | 1.000 | 0.000     |
| Naive D-1          | Benchmark        | purchases | SUD  | 112.160  | 160.489  | 0.056 | 0.080 | 1.050 | -4.979    |
| XGBoost            | Machine learning | purchases | CALA | 14.071   | 21.174   | 0.022 | 0.034 | 0.479 | 52.144    |
| LEAR               | Statistical      | purchases | CALA | 15.061   | 23.003   | 0.024 | 0.036 | 0.512 | 48.778    |
| TCN recursive 24m  | Deep learning    | purchases | CALA | 15.948   | 22.575   | 0.025 | 0.036 | 0.542 | 45.761    |
| Random Forest      | Machine learning | purchases | CALA | 16.778   | 24.655   | 0.027 | 0.039 | 0.571 | 42.938    |
| LSTM base          | Deep learning    | purchases | CALA | 17.500   | 24.029   | 0.028 | 0.038 | 0.595 | 40.484    |
| LSTM rolling 24m   | Deep learning    | purchases | CALA | 18.445   | 24.864   | 0.029 | 0.039 | 0.627 | 37.267    |
| TCN base           | Deep learning    | purchases | CALA | 18.953   | 25.440   | 0.030 | 0.040 | 0.645 | 35.541    |
| LSTM recursive 24m | Deep learning    | purchases | CALA | 19.631   | 26.771   | 0.031 | 0.042 | 0.668 | 33.236    |
| TCN rolling 24m    | Deep learning    | purchases | CALA | 19.928   | 28.641   | 0.032 | 0.045 | 0.678 | 32.224    |
| Naive D-1          | Benchmark        | purchases | CALA | 25.764   | 39.128   | 0.041 | 0.062 | 0.876 | 12.376    |
| SARIMAX            | Statistical      | purchases | CALA | 29.028   | 44.148   | 0.046 | 0.070 | 0.987 | 1.273     |
| Naive D-7          | Benchmark        | purchases | CALA | 29.403   | 44.876   | 0.047 | 0.071 | 1.000 | 0.000     |
| XGBoost            | Machine learning | purchases | SICI | 47.787   | 71.969   | 0.026 | 0.039 | 0.557 | 44.278    |
| LEAR               | Statistical      | purchases | SICI | 52.091   | 78.248   | 0.028 | 0.042 | 0.607 | 39.260    |
| LSTM recursive 24m | Deep learning    | purchases | SICI | 52.731   | 77.101   | 0.028 | 0.041 | 0.615 | 38.513    |
| Random Forest      | Machine learning | purchases | SICI | 55.270   | 80.313   | 0.030 | 0.043 | 0.644 | 35.552    |
| TCN recursive 24m  | Deep learning    | purchases | SICI | 60.659   | 83.599   | 0.032 | 0.045 | 0.707 | 29.269    |
| TCN base           | Deep learning    | purchases | SICI | 63.471   | 85.196   | 0.034 | 0.046 | 0.740 | 25.990    |
| LSTM base          | Deep learning    | purchases | SICI | 63.725   | 88.071   | 0.034 | 0.047 | 0.743 | 25.694    |
| TCN rolling 24m    | Deep learning    | purchases | SICI | 65.747   | 90.523   | 0.035 | 0.048 | 0.767 | 23.336    |
| LSTM rolling 24m   | Deep learning    | purchases | SICI | 69.856   | 94.039   | 0.037 | 0.050 | 0.815 | 18.545    |
| SARIMAX            | Statistical      | purchases | SICI | 84.773   | 118.924  | 0.045 | 0.064 | 0.989 | 1.151     |
| Naive D-7          | Benchmark        | purchases | SICI | 85.760   | 130.673  | 0.046 | 0.070 | 1.000 | 0.000     |
| Naive D-1          | Benchmark        | purchases | SICI | 87.693   | 128.611  | 0.047 | 0.069 | 1.022 | -2.255    |
| XGBoost            | Machine learning | purchases | SARD | 44.131   | 84.836   | 0.047 | 0.091 | 0.668 | 33.215    |
| LEAR               | Statistical      | purchases | SARD | 46.014   | 86.164   | 0.049 | 0.092 | 0.696 | 30.365    |
| TCN recursive 24m  | Deep learning    | purchases | SARD | 49.862   | 98.970   | 0.053 | 0.106 | 0.755 | 24.542    |
| Random Forest      | Machine learning | purchases | SARD | 50.499   | 97.000   | 0.054 | 0.104 | 0.764 | 23.578    |
| LSTM recursive 24m | Deep learning    | purchases | SARD | 51.717   | 102.626  | 0.055 | 0.110 | 0.783 | 21.735    |
| LSTM base          | Deep learning    | purchases | SARD | 54.975   | 101.867  | 0.059 | 0.109 | 0.832 | 16.804    |
| TCN base           | Deep learning    | purchases | SARD | 55.855   | 102.128  | 0.060 | 0.109 | 0.845 | 15.473    |
| Naive D-1          | Benchmark        | purchases | SARD | 56.947   | 104.298  | 0.061 | 0.112 | 0.862 | 13.821    |
| LSTM rolling 24m   | Deep learning    | purchases | SARD | 57.430   | 104.593  | 0.061 | 0.112 | 0.869 | 13.090    |
| SARIMAX            | Statistical      | purchases | SARD | 61.885   | 98.230   | 0.066 | 0.105 | 0.936 | 6.347     |
| Naive D-7          | Benchmark        | purchases | SARD | 66.079   | 114.210  | 0.071 | 0.122 | 1.000 | 0.000     |
| TCN rolling 24m    | Deep learning    | purchases | SARD | 68.999   | 119.550  | 0.074 | 0.128 | 1.044 | -4.419    |
| XGBoost            | Machine learning | sales     | NORD | 1040.313 | 1371.294 | 0.076 | 0.100 | 0.626 | 37.352    |
| TCN base           | Deep learning    | sales     | NORD | 1064.842 | 1384.652 | 0.078 | 0.101 | 0.641 | 35.875    |
| LEAR               | Statistical      | sales     | NORD | 1081.836 | 1421.584 | 0.079 | 0.104 | 0.651 | 34.852    |
| Random Forest      | Machine learning | sales     | NORD | 1100.176 | 1465.220 | 0.080 | 0.107 | 0.662 | 33.748    |
| LSTM base          | Deep learning    | sales     | NORD | 1163.234 | 1500.697 | 0.085 | 0.109 | 0.701 | 29.950    |
| TCN recursive 24m  | Deep learning    | sales     | NORD | 1284.481 | 1722.533 | 0.094 | 0.126 | 0.773 | 22.649    |
| LSTM recursive 24m | Deep learning    | sales     | NORD | 1288.412 | 1686.385 | 0.094 | 0.123 | 0.776 | 22.412    |
| LSTM rolling 24m   | Deep learning    | sales     | NORD | 1397.837 | 1838.228 | 0.102 | 0.134 | 0.842 | 15.822    |
| TCN rolling 24m    | Deep learning    | sales     | NORD | 1419.420 | 2163.602 | 0.104 | 0.158 | 0.855 | 14.523    |
| Naive D-7          | Benchmark        | sales     | NORD | 1660.582 | 2182.304 | 0.121 | 0.159 | 1.000 | 0.000     |
| SARIMAX            | Statistical      | sales     | NORD | 1672.517 | 2395.739 | 0.122 | 0.175 | 1.007 | -0.719    |
| Naive D-1          | Benchmark        | sales     | NORD | 1811.882 | 2628.129 | 0.132 | 0.192 | 1.091 | -9.111    |
| XGBoost            | Machine learning | sales     | CNOR | 133.511  | 177.005  | 0.078 | 0.104 | 0.694 | 30.566    |
| LEAR               | Statistical      | sales     | CNOR | 134.539  | 177.078  | 0.079 | 0.104 | 0.700 | 30.031    |
| TCN base           | Deep learning    | sales     | CNOR | 143.363  | 183.697  | 0.084 | 0.107 | 0.746 | 25.442    |
| LSTM recursive 24m | Deep learning    | sales     | CNOR | 147.123  | 188.358  | 0.086 | 0.110 | 0.765 | 23.487    |

| Model              | Family           | Target | Zone | MAE     | RMSE    | NMAE  | NRMSE | rMAE  | impMAE(%) |
|--------------------|------------------|--------|------|---------|---------|-------|-------|-------|-----------|
| Random Forest      | Machine learning | sales  | CNOR | 148.232 | 188.747 | 0.087 | 0.110 | 0.771 | 22.910    |
| LSTM base          | Deep learning    | sales  | CNOR | 148.935 | 189.118 | 0.087 | 0.111 | 0.775 | 22.544    |
| TCN recursive 24m  | Deep learning    | sales  | CNOR | 152.984 | 196.819 | 0.090 | 0.115 | 0.796 | 20.439    |
| Naive D-1          | Benchmark        | sales  | CNOR | 154.745 | 223.803 | 0.090 | 0.131 | 0.805 | 19.523    |
| LSTM rolling 24m   | Deep learning    | sales  | CNOR | 156.574 | 197.884 | 0.092 | 0.116 | 0.814 | 18.572    |
| TCN rolling 24m    | Deep learning    | sales  | CNOR | 157.451 | 200.027 | 0.092 | 0.117 | 0.819 | 18.116    |
| SARIMAX            | Statistical      | sales  | CNOR | 180.113 | 231.450 | 0.105 | 0.135 | 0.937 | 6.330     |
| Naive D-7          | Benchmark        | sales  | CNOR | 192.285 | 255.695 | 0.112 | 0.150 | 1.000 | 0.000     |
| XGBoost            | Machine learning | sales  | CSUD | 409.545 | 522.770 | 0.120 | 0.154 | 0.702 | 29.829    |
| LEAR               | Statistical      | sales  | CSUD | 416.541 | 538.002 | 0.122 | 0.158 | 0.714 | 28.630    |
| Random Forest      | Machine learning | sales  | CSUD | 435.596 | 562.663 | 0.128 | 0.165 | 0.746 | 25.365    |
| TCN recursive 24m  | Deep learning    | sales  | CSUD | 458.024 | 597.830 | 0.135 | 0.176 | 0.785 | 21.522    |
| TCN base           | Deep learning    | sales  | CSUD | 469.356 | 594.544 | 0.138 | 0.175 | 0.804 | 19.581    |
| LSTM base          | Deep learning    | sales  | CSUD | 504.241 | 641.061 | 0.148 | 0.188 | 0.864 | 13.604    |
| SARIMAX            | Statistical      | sales  | CSUD | 517.046 | 687.121 | 0.152 | 0.202 | 0.886 | 11.410    |
| TCN rolling 24m    | Deep learning    | sales  | CSUD | 545.758 | 768.846 | 0.160 | 0.226 | 0.935 | 6.490     |
| LSTM rolling 24m   | Deep learning    | sales  | CSUD | 547.202 | 691.355 | 0.161 | 0.203 | 0.938 | 6.243     |
| LSTM recursive 24m | Deep learning    | sales  | CSUD | 549.842 | 698.060 | 0.162 | 0.205 | 0.942 | 5.790     |
| Naive D-1          | Benchmark        | sales  | CSUD | 572.709 | 777.479 | 0.168 | 0.228 | 0.981 | 1.872     |
| Naive D-7          | Benchmark        | sales  | CSUD | 583.637 | 753.098 | 0.172 | 0.221 | 1.000 | 0.000     |
| TCN base           | Deep learning    | sales  | SUD  | 375.143 | 473.436 | 0.113 | 0.142 | 0.483 | 51.733    |
| LSTM base          | Deep learning    | sales  | SUD  | 386.541 | 484.439 | 0.116 | 0.146 | 0.497 | 50.267    |
| Random Forest      | Machine learning | sales  | SUD  | 412.256 | 526.462 | 0.124 | 0.158 | 0.530 | 46.958    |
| XGBoost            | Machine learning | sales  | SUD  | 414.598 | 526.985 | 0.125 | 0.159 | 0.533 | 46.657    |
| LSTM rolling 24m   | Deep learning    | sales  | SUD  | 419.300 | 527.314 | 0.126 | 0.159 | 0.539 | 46.052    |
| LEAR               | Statistical      | sales  | SUD  | 420.154 | 529.214 | 0.126 | 0.159 | 0.541 | 45.942    |
| TCN recursive 24m  | Deep learning    | sales  | SUD  | 471.100 | 596.139 | 0.142 | 0.180 | 0.606 | 39.387    |
| SARIMAX            | Statistical      | sales  | SUD  | 496.477 | 639.475 | 0.150 | 0.192 | 0.639 | 36.122    |
| TCN rolling 24m    | Deep learning    | sales  | SUD  | 497.186 | 784.322 | 0.150 | 0.236 | 0.640 | 36.031    |
| LSTM recursive 24m | Deep learning    | sales  | SUD  | 503.516 | 648.260 | 0.152 | 0.195 | 0.648 | 35.216    |
| Naive D-1          | Benchmark        | sales  | SUD  | 650.240 | 837.828 | 0.196 | 0.252 | 0.837 | 16.338    |
| Naive D-7          | Benchmark        | sales  | SUD  | 777.226 | 992.299 | 0.234 | 0.299 | 1.000 | 0.000     |
| XGBoost            | Machine learning | sales  | CALA | 310.195 | 403.556 | 0.216 | 0.281 | 0.663 | 33.668    |
| TCN base           | Deep learning    | sales  | CALA | 315.257 | 401.183 | 0.219 | 0.279 | 0.674 | 32.586    |
| Random Forest      | Machine learning | sales  | CALA | 329.194 | 428.467 | 0.229 | 0.298 | 0.704 | 29.605    |
| LEAR               | Statistical      | sales  | CALA | 329.283 | 421.042 | 0.229 | 0.293 | 0.704 | 29.586    |
| TCN recursive 24m  | Deep learning    | sales  | CALA | 334.117 | 433.029 | 0.232 | 0.301 | 0.715 | 28.553    |
| LSTM base          | Deep learning    | sales  | CALA | 342.134 | 437.038 | 0.238 | 0.304 | 0.732 | 26.838    |
| LSTM rolling 24m   | Deep learning    | sales  | CALA | 357.940 | 464.699 | 0.249 | 0.323 | 0.765 | 23.458    |
| LSTM recursive 24m | Deep learning    | sales  | CALA | 372.216 | 482.749 | 0.259 | 0.336 | 0.796 | 20.406    |
| TCN rolling 24m    | Deep learning    | sales  | CALA | 373.037 | 510.827 | 0.260 | 0.355 | 0.798 | 20.230    |
| Naive D-1          | Benchmark        | sales  | CALA | 389.048 | 535.162 | 0.271 | 0.372 | 0.832 | 16.806    |
| SARIMAX            | Statistical      | sales  | CALA | 418.969 | 529.756 | 0.292 | 0.369 | 0.896 | 10.408    |
| Naive D-7          | Benchmark        | sales  | CALA | 467.641 | 611.810 | 0.325 | 0.426 | 1.000 | 0.000     |
| Random Forest      | Machine learning | sales  | SICI | 245.171 | 314.226 | 0.179 | 0.230 | 0.637 | 36.254    |
| XGBoost            | Machine learning | sales  | SICI | 247.066 | 322.106 | 0.181 | 0.235 | 0.642 | 35.762    |
| TCN base           | Deep learning    | sales  | SICI | 249.297 | 314.711 | 0.182 | 0.230 | 0.648 | 35.182    |
| LSTM recursive 24m | Deep learning    | sales  | SICI | 254.052 | 326.472 | 0.186 | 0.239 | 0.660 | 33.945    |
| LEAR               | Statistical      | sales  | SICI | 254.261 | 328.909 | 0.186 | 0.240 | 0.661 | 33.891    |
| SARIMAX            | Statistical      | sales  | SICI | 260.575 | 348.380 | 0.190 | 0.255 | 0.677 | 32.249    |
| LSTM base          | Deep learning    | sales  | SICI | 264.695 | 335.426 | 0.193 | 0.245 | 0.688 | 31.178    |
| TCN rolling 24m    | Deep learning    | sales  | SICI | 266.003 | 334.454 | 0.194 | 0.244 | 0.692 | 30.838    |
| TCN recursive 24m  | Deep learning    | sales  | SICI | 269.152 | 344.193 | 0.197 | 0.252 | 0.700 | 30.019    |
| LSTM rolling 24m   | Deep learning    | sales  | SICI | 270.143 | 342.259 | 0.197 | 0.250 | 0.702 | 29.762    |
| Naive D-1          | Benchmark        | sales  | SICI | 297.438 | 404.195 | 0.217 | 0.295 | 0.773 | 22.665    |
| Naive D-7          | Benchmark        | sales  | SICI | 384.609 | 510.283 | 0.281 | 0.373 | 1.000 | 0.000     |
| TCN base           | Deep learning    | sales  | SARD | 117.132 | 159.592 | 0.082 | 0.112 | 0.572 | 42.835    |
| XGBoost            | Machine learning | sales  | SARD | 125.681 | 165.000 | 0.089 | 0.116 | 0.613 | 38.663    |
| TCN recursive 24m  | Deep learning    | sales  | SARD | 129.182 | 178.271 | 0.091 | 0.126 | 0.630 | 36.954    |
| LEAR               | Statistical      | sales  | SARD | 130.518 | 169.062 | 0.092 | 0.119 | 0.637 | 36.302    |
| Random Forest      | Machine learning | sales  | SARD | 130.567 | 173.822 | 0.092 | 0.122 | 0.637 | 36.278    |
| LSTM base          | Deep learning    | sales  | SARD | 135.599 | 176.068 | 0.096 | 0.124 | 0.662 | 33.822    |

| Model              | Family        | Target | Zone | MAE     | RMSE    | NMAE  | NRMSE | rMAE  | impMAE(%) |
|--------------------|---------------|--------|------|---------|---------|-------|-------|-------|-----------|
| SARIMAX            | Statistical   | sales  | SARD | 136.669 | 181.390 | 0.096 | 0.128 | 0.667 | 33.300    |
| LSTM recursive 24m | Deep learning | sales  | SARD | 144.163 | 200.551 | 0.102 | 0.141 | 0.704 | 29.643    |
| TCN rolling 24m    | Deep learning | sales  | SARD | 145.180 | 188.546 | 0.102 | 0.133 | 0.709 | 29.146    |
| LSTM rolling 24m   | Deep learning | sales  | SARD | 151.979 | 193.658 | 0.107 | 0.136 | 0.742 | 25.828    |
| Naive D-1          | Benchmark     | sales  | SARD | 157.793 | 216.804 | 0.111 | 0.153 | 0.770 | 22.991    |
| Naive D-7          | Benchmark     | sales  | SARD | 204.902 | 272.490 | 0.144 | 0.192 | 1.000 | 0.000     |